

SPIS TREŚCI

1	Wstęp	5
1.1	Cel pracy	5
1.2	Zakres pracy	5
1.3	Problem badawczy	6
1.4	Struktura pracy	7
2	Podstawy teoretyczne	9
2.1	Poker jako gra z niepełną informacją	9
2.2	Texas Hold'em	9
2.3	Ranking układów pokerowych	9
2.4	Ultimate Texas Hold'em	10
2.4.1	Charakterystyka gry	10
2.4.2	Różnice względem klasycznego Texas Hold'em	10
2.4.3	Przebieg rozdania	11
2.4.4	Zakłady i decyzje gracza	11
2.4.5	Rozstrzyganie rozdania	11
2.4.6	Zakłady poboczne	12
2.4.7	Tabele wypłat	12
2.5	Wartość oczekiwana, house edge i strategia bazowa	13
2.6	Uczenie przez wzmacnianie	14
3	Projekt i implementacja środowiska Ultimate Texas Hold'em	17
3.1	Założenia projektowe środowiska	17
3.2	Architektura rozwiązania	18
3.2.1	Przepływ danych podczas wykonania akcji	20
3.3	Model kart i sterowanie talią	21
3.3.1	Reprezentacja karty	21
3.3.2	Standardowa talia	21
3.3.3	Abstrakcja źródła kart	22
3.3.4	Deterministyczna talia testowa	22
3.3.5	Powtarzalność losowych rozdań	23
3.4	Ocena układów pokerowych	23

3.4.1	Reprezentacja wartości ręki	24
3.4.2	Ocena układu pięciokartowego	24
3.4.3	Obsługa niskiego strita z asem	25
3.4.4	Wybór najlepszej ręki	26
3.5	Stan wewnętrzny i obserwacja agenta	26
3.5.1	Pełny stan rozdania	27
3.5.2	Obserwacja agenta	28
3.5.3	Projekcja stanu na obserwację	29
3.5.4	Ochrona przed wyciekiem informacji	30
3.6	Przebieg rozdania i maszyna stanów	31
3.6.1	Rozpoczęcie rozdania	31
3.6.2	Faza preflop	32
3.6.3	Faza flopa	32
3.6.4	Faza rivera	32
3.6.5	Faza terminalna	33
3.6.6	Tabela przejść	33
3.6.7	Diagram maszyny stanów	33
3.6.8	Zagwarantowanie pojedynczego zakładu Play	33
3.7	Przestrzeń akcji i walidacja decyzji	34
3.7.1	Akcje legalne jako część obserwacji	35
3.7.2	Walidacja decyzji	36
3.7.3	Niezmienniki przestrzeni akcji	36
3.7.4	Planowane odwzorowanie na przestrzeń Gymnasium	37
3.8	Showdown i rozliczanie zakładów	37
3.8.1	Przebieg showdownu	37
3.8.2	Kwalifikacja krupiera	38
3.8.3	Model rozliczenia zakładów	39
3.8.4	Rozliczenie zakładu Blind	40
3.8.5	Rozliczenie Ante i Play	41
3.8.6	Rozliczenie spasowania	41
3.8.7	Przykładowe rozliczenia	41
3.9	Funkcja nagrody i wynik epizodu	42
3.9.1	Rozliczenie finansowe a nagroda	42
3.9.2	Podstawowa nagroda terminalna	43
3.9.3	Zwrot epizodyczny	43
3.9.4	Brak nagrody za nielegalną akcję	44
3.9.5	Możliwość stosowania alternatywnych funkcji nagrody	44
3.10	Interfejs silnika i integracja z Gymnasium	45
3.10.1	Tworzenie silnika	45

3.10.2	Rozpoczęcie epizodu	46
3.10.3	Wykonanie decyzji	46
3.10.4	Właściwości publiczne	47
3.10.5	Rola adaptera Gymnasium	47
3.10.6	Odwzorowanie metody reset	48
3.10.7	Odwzorowanie metody step	49
3.10.8	Planowana przestrzeń akcji	49
3.10.9	Planowana przestrzeń obserwacji	50
3.10.10	Dane pomocnicze	50
3.10.11	Porównanie interfejsów	51
3.11	Rejestrowanie przebiegu i walidacja środowiska	51
3.11.1	Opcjonalne rejestrowanie historii	51
3.11.2	Rekord pojedynczej decyzji	52
3.11.3	Ślad całego rozdania	53
3.11.4	Transakcyjne zapisywanie decyzji	53
3.11.5	Testy deterministyczne	54
3.11.6	Poziomy testowania	55
3.11.7	Walidowane niezmienniki	55
3.11.8	Pokrycie kodu	56

1. Wstęp

Gry karciane stanowią interesujący obszar zastosowań metod sztucznej inteligencji, ponieważ łączą element losowości, niepełnej informacji oraz sekwencyjnego podejmowania decyzji. W przeciwieństwie do wielu klasycznych problemów uczenia maszynowego, w których model uczy się na podstawie gotowego zbioru danych, w grach agent musi samodzielnie podejmować decyzje, obserwować ich konsekwencje i stopniowo poprawiać swoją strategię.

Szczególnie interesującym przypadkiem są gry pokerowe, w których wynik pojedynczego rozdania zależy nie tylko od jakości posiadanych kart, ale również od decyzji podejmowanych w kolejnych etapach gry. Ultimate Texas Hold'em jest kasynową odmianą pokera opartą na zasadach Texas Hold'em, w której gracz rywalizuje z krupierem, a nie z innymi graczami. Gra ta posiada ograniczoną liczbę możliwych akcji, jasno określone reguły wypłat oraz sekwencyjny przebieg rozdania, co czyni ją odpowiednim środowiskiem do modelowania jako problem uczenia przez wzmacnianie.

Uczenie przez wzmacnianie (ang. *reinforcement learning*, RL) pozwala analizować sytuacje, w których agent uczy się strategii poprzez interakcję ze środowiskiem. W przypadku Ultimate Texas Hold'em środowiskiem może być symulator gry, stanem aktualna sytuacja w rozdaniu, akcją decyzja gracza, natomiast nagrodą końcowy wynik finansowy rozdania. Takie podejście umożliwia badanie, w jaki sposób różne reprezentacje stanu oraz różne definicje funkcji nagrody wpływają na jakość wyuczonej strategii.

1.1. Cel pracy

Głównym celem pracy jest porównanie wybranych metod uczenia przez wzmacnianie w zadaniu uczenia strategii gry w Ultimate Texas Hold'em. Szczególny nacisk położony zostanie na analizę wpływu reprezentacji stanu oraz funkcji nagrody na jakość strategii uzyskiwanej przez agenta.

Celem praktycznym pracy jest implementacja środowiska symulującego rozgrywkę w Ultimate Texas Hold'em oraz przygotowanie agentów podejmujących decyzje w kolejnych etapach rozdania. Uzyskane strategie zostaną porównane ze strategiami bazowymi, takimi jak agent losowy oraz prosty agent regułowy.

1.2. Zakres pracy

Zakres pracy obejmuje zaprojektowanie i implementację uproszczonego środowiska gry Ultimate Texas Hold'em, w którym uwzględnione zostaną podstawowe elementy rozgrywki: karty gracza i krupiera, karty wspólne, obowiązkowe zakłady Ante i Blind oraz

decyzyjny zakład Play. W podstawowej wersji środowiska pominięte zostaną zakłady poboczne, takie jak Trips oraz Progressive, ponieważ nie są one bezpośrednio związane z głównym procesem decyzyjnym agenta.

W ramach pracy zostaną przygotowane różne warianty reprezentacji stanu gry. Reprezentacja stanu może obejmować między innymi informacje o kartach gracza, kartach wspólnych, aktualnym etapie rozdania oraz dostępnych akcjach. Analizie podlegać będzie również sposób definiowania funkcji nagrody, w szczególności to, czy agent otrzymuje nagrodę wyłącznie na końcu rozdania, czy także dodatkowe informacje pośrednie wspierające proces uczenia.

Założenia niniejszej pracy są następujące:

1. Implementacja środowiska symulującego rozgrywkę w Ultimate Texas Hold'em zgodnie z podstawowymi zasadami gry.
2. Implementacja mechanizmu oceny układów pokerowych oraz rozstrzygania wyniku rozdania pomiędzy graczem a krupierem.
3. Przygotowanie agentów bazowych, w szczególności agenta losowego oraz prostego agenta regułowego.
4. Implementacja i porównanie wybranych metod uczenia przez wzmacnianie w zadaniu podejmowania decyzji w Ultimate Texas Hold'em.
5. Analiza wpływu różnych reprezentacji stanu na jakość wyuczonej strategii.
6. Analiza wpływu różnych funkcji nagrody na stabilność procesu uczenia oraz końcowy wynik agenta.
7. Ocena agentów z wykorzystaniem metryk takich jak średni wynik na rozdanie, wartość oczekiwana strategii oraz porównanie względem strategii bazowych.

1.3. Problem badawczy

Problem badawczy rozważany w pracy można sformułować następująco: w jaki sposób wybór reprezentacji stanu oraz funkcji nagrody wpływa na skuteczność metod uczenia przez wzmacnianie w grze Ultimate Texas Hold'em?

W pracy analizowane będzie, czy agent uczony metodami reinforcement learning jest w stanie uzyskać strategię lepszą od strategii losowej oraz prostych heurystyk. Ze względu na konstrukcję gier kasynowych celem nie jest wykazanie, że agent może trwale osiągnąć dodatnią wartość oczekiwaną przeciwko kasynu, lecz sprawdzenie, czy możliwe jest nauczenie strategii minimalizującej oczekiwaną stratę.

1.4. Struktura pracy

Praca składa się z kilku rozdziałów. W rozdziale pierwszym przedstawiono cel, zakres oraz problem badawczy pracy. Rozdział drugi zawiera podstawy teoretyczne dotyczące pokera, Texas Hold'em, Ultimate Texas Hold'em, wartości oczekiwanej, przewagi kasyna oraz uczenia przez wzmacnianie.

W kolejnych rozdziałach opisany zostanie projekt środowiska symulacyjnego, sposób reprezentacji stanu gry, funkcja nagrody oraz zastosowane metody uczenia przez wzmacnianie. Następnie przedstawione zostaną eksperymenty, uzyskane wyniki oraz analiza porównawcza agentów. Ostatni rozdział będzie zawierał podsumowanie pracy, wnioski oraz możliwe kierunki dalszego rozwoju.

2. Podstawy teoretyczne

2.1. Poker jako gra z niepełną informacją

Poker jest rodziną gier karcianych, w których wynik zależy zarówno od losowego rozdania kart, jak i od decyzji podejmowanych przez graczy. W przeciwieństwie do gier z pełną informacją, takich jak szachy, uczestnicy rozgrywki nie znają wszystkich elementów stanu gry. Ukryte pozostają przede wszystkim karty przeciwników, co sprawia, że decyzje podejmowane są w warunkach niepewności.

Z punktu widzenia sztucznej inteligencji poker jest interesującym problemem badawczym, ponieważ łączy losowość, niepełną informację, sekwencyjne podejmowanie decyzji oraz konieczność maksymalizacji wyniku w długim horyzoncie czasowym. W takim środowisku pojedyncza decyzja nie powinna być oceniana wyłącznie przez pryzmat wyniku jednego rozdania, lecz przez jej wpływ na oczekiwaną wartość strategii w wielu powtórzeniach gry.

2.2. Texas Hold'em

Texas Hold'em jest jedną z najpopularniejszych odmian pokera. Każdy gracz otrzymuje dwie zakryte karty własne (ang. *hole cards*), natomiast na stole stopniowo odkrywanych jest pięć kart wspólnych (ang. *community cards*). Gracz tworzy najlepszy możliwy układ pięciokartowy, wykorzystując dowolną kombinację swoich dwóch kart oraz pięciu kart wspólnych.

Rozgrywka w klasycznym Texas Hold'em składa się z kilku etapów: etapu przed flopem (ang. *preflop*), flopa (ang. *flop*), czyli odkrycia trzech pierwszych kart wspólnych, turna (ang. *turn*), czyli czwartej karty wspólnej, rivera (ang. *river*), czyli piątej i ostatniej karty wspólnej, oraz showdownu (ang. *showdown*), czyli końcowego porównania układów. W trakcie kolejnych rund licytacji gracze mogą podejmować decyzje takie jak czekanie (ang. *check*), postawienie zakładu (ang. *bet*), sprawdzenie zakładu przeciwnika (ang. *call*), podbicie (ang. *raise*) lub spasowanie (ang. *fold*). W odmianach no-limit możliwe jest również stosowanie różnych rozmiarów zakładów, co istotnie zwiększa przestrzeń możliwych strategii.

2.3. Ranking układów pokerowych

W Texas Hold'em oraz Ultimate Texas Hold'em wynik rozdania zależy od siły najlepszego układu pięciokartowego utworzonego z dostępnych kart. Układy pokerowe mają ustaloną hierarchię, która pozwala jednoznacznie porównać rękę gracza i krupiera

podczas showdownu. Przykładowy ranking układów od najsilniejszego do najsłabszego przedstawiono w tabeli 2.1 [1].

Najsilniejszym układem jest poker królewski, natomiast najsłabszym układem jest wysoka karta. W przypadku gdy dwóch uczestników rozdania posiada układ tej samej kategorii, o zwycięstwie decydują wartości kart wchodzących w skład układu oraz karty dodatkowe, określane jako kickery.

Tabela 2.1. Ranking układów pokerowych od najsilniejszego do najsłabszego

Układ	Opis
Poker królewski	A, K, Q, J, 10 w tym samym kolorze
Poker	Pięć kolejnych kart w tym samym kolorze
Kareta	Cztery karty tej samej wartości
Full	Trójka oraz para
Kolor	Pięć kart w tym samym kolorze
Strit	Pięć kolejnych kart
Trójka	Trzy karty tej samej wartości
Dwie pary	Dwa różne układy par
Para	Dwie karty tej samej wartości
Wysoka karta	Brak silniejszego układu

2.4. Ultimate Texas Hold'em

2.4.1. Charakterystyka gry

Ultimate Texas Hold'em jest kasynową odmianą pokera opartą na zasadach Texas Hold'em. W przeciwieństwie do klasycznej odmiany wieloosobowej gracz nie rywalizuje z innymi graczami, lecz z krupierem reprezentującym kasyno. Zarówno gracz, jak i krupier otrzymują po dwie karty własne, a następnie korzystają z pięciu kart wspólnych w celu utworzenia najlepszego możliwego układu pięciokartowego [1], [2].

2.4.2. Różnice względem klasycznego Texas Hold'em

Najważniejsza różnica względem klasycznego Texas Hold'em polega na tym, że w Ultimate Texas Hold'em gracz podejmuje decyzje przeciwko kasynu, a nie przeciwko innym graczom. Nie występuje więc klasyczna rywalizacja między wieloma uczestnikami stołu, nie ma konieczności blefowania przeciwników ani reagowania na ich indywidualne style gry.

Istotną cechą Ultimate Texas Hold'em jest również ograniczona liczba momentów decyzyjnych. Gracz może wykonać zakład Play (ang. *Play bet*) tylko raz w trakcie rozdania: przed flopem, po flopie albo po odkryciu wszystkich kart wspólnych. Im wcześniej gracz zdecyduje się na zagranie agresywne, tym większy zakład może postawić, ale decyzja jest wtedy podejmowana przy mniejszej liczbie informacji [1], [3].

Tabela 2.2. Porównanie Texas Hold'em i Ultimate Texas Hold'em

Cecha	Texas Hold'em	Ultimate Texas Hold'em
Przeciwnik	inni gracze	krupier / kasyno
Liczba graczy	wielu	jeden gracz przeciwko dealerowi
Zakłady	zmienne	ustalone mnożniki Ante
Blefowanie	istotne	brak klasycznego blefowania
Liczba decyzji	duża	ograniczona

2.4.3. Przebieg rozdania

Rozdanie rozpoczyna się od postawienia przez gracza obowiązkowych zakładów Ante (ang. *Ante bet*) oraz Blind (ang. *Blind bet*). Zakłady te mają zwykle taką samą wartość. Opcjonalnie gracz może również postawić zakład poboczny (ang. *side bet*), na przykład Trips, który jest rozliczany niezależnie od wyniku pojedynku z krupierem [1].

Po wniesieniu zakładów gracz oraz krupier otrzymują po dwie zakryte karty własne. Następnie gracz podejmuje pierwszą decyzję. Może zaczekać albo wykonać zakład Play w wysokości trzykrotności lub czterokrotności zakładu Ante.

Jeżeli gracz nie wykona zakładu Play przed flopem, odkrywane są trzy pierwsze karty wspólne, czyli flop. Na tym etapie gracz może ponownie zaczekać albo wykonać zakład Play w wysokości dwukrotności Ante. Jeżeli również po flopie gracz zdecyduje się zaczekać, odkrywane są dwie ostatnie karty wspólne, czyli turn oraz river. Wtedy gracz musi wybrać pomiędzy spasowaniem a wykonaniem zakładu Play w wysokości jednokrotności Ante [1], [3].

2.4.4. Zakłady i decyzje gracza

W podstawowym modelu gry występują trzy główne rodzaje zakładów: Ante, Blind oraz Play. Zakłady Ante i Blind są obowiązkowe i stawiane przed rozpoczęciem rozdania. Zakład Play jest natomiast bezpośrednio związany z decyzją gracza i może zostać wykonany tylko raz w trakcie rozdania.

Z punktu widzenia modelowania problemu jako środowiska uczenia przez wzmocnienie najważniejszy jest zakład Play, ponieważ jego wysokość i moment wykonania wynikają z decyzji agenta. Ante i Blind można traktować jako początkowy koszt udziału w rozdaniu, natomiast Play odzwierciedla wybór strategii w aktualnym stanie gry.

Dostępne decyzje gracza w poszczególnych etapach rozdania przedstawiono w tabeli 2.3.

2.4.5. Rozstrzygnięcie rozdania

Po zakończeniu etapu decyzyjnego krupier odsłania swoje dwie karty własne. Zarówno gracz, jak i krupier tworzą najlepszy możliwy układ pięciokartowy z siedmiu

Tabela 2.3. Dostępne decyzje gracza w Ultimate Texas Hold'em

Etap	Dostępne akcje	Wysokość zakładu Play
Preflop	check / raise	3x lub 4x Ante
Flop	check / raise	2x Ante
River	fold / raise	1x Ante

dostępnych kart: dwóch kart własnych oraz pięciu kart wspólnych. Następnie układy są porównywane zgodnie ze standardowym rankingiem układów pokerowych.

Krupier kwalifikuje się do gry, jeżeli posiada co najmniej parę. Jeżeli krupier się nie kwalifikuje, zakład Ante jest zwracany graczowi. Zakład Play jest rozliczany na podstawie bezpośredniego porównania układu gracza i krupiera. W przypadku zwycięstwa gracza Play wypłacany jest w stosunku 1:1, w przypadku przegranej gracz traci zakład Play, a w przypadku remisu zakład jest zwracany [1], [3].

Zakład Blind ma odrębną logikę rozliczania. Jeżeli gracz pokona krupiera układem o wartości co najmniej strita, Blind wypłacany jest zgodnie z tabelą wypłat. Jeżeli gracz wygra rozdanie układem słabszym niż strit, zakład Blind jest zwracany. W przypadku przegranej gracza zakład Blind jest tracony, a w przypadku remisu zwracany [1].

2.4.6. Zakłady poboczne

W wielu wariantach Ultimate Texas Hold'em dostępne są również zakłady poboczne, takie jak Trips lub Progressive. Zakład Trips jest opcjonalny i wypłacany na podstawie końcowego układu gracza, niezależnie od tego, czy gracz pokonał krupiera. Najczęściej wygrywa on wtedy, gdy końcowy układ gracza ma wartość co najmniej trójki [1].

Zakłady poboczne mogą różnić się w zależności od kasyna i stosowanej tabeli wypłat.

TODO: Decyzja do potwierdzenia z promotorem: czy w podstawowym środowisku uwzględnić za

W podstawowej wersji środowiska zakłady poboczne Trips oraz Progressive nie będą uwzględniane. Wynika to z faktu, że głównym celem pracy jest modelowanie sekwencyjnego procesu decyzyjnego dotyczącego zakładu Play. Zakłady poboczne są opcjonalne, zależą od tabel wypłat konkretnego kasyna i zwiększają wariancję wyniku, nie zmieniając podstawowej struktury decyzji: check, raise albo fold.

2.4.7. Tabele wypłat

Zakład Blind nie jest wypłacany dla każdego zwycięskiego układu gracza. Zgodnie z tabelą wypłat, bonus na zakładzie Blind wypłacany jest wtedy, gdy gracz pokona krupiera i posiada układ o wartości co najmniej strita. Jeżeli gracz pokona krupiera układem słabszym niż strit, zakład Blind jest zwracany. Tabelę wypłat zakładu Blind

przedstawiono w tabeli 2.4 [1].

Tabela 2.4. Tabela wypłat zakładu Trips przyjęta w modelu środowiska

Układ gracza	Wypłata
Poker królewski	500:1
Poker	50:1
Kareta	10:1
Full	3:1
Kolor	3:2
Strit	1:1
Pozostałe układy	Zwrot

Zakład Trips jest opcjonalnym zakładem pobocznym. Jest on rozliczany na podstawie końcowego układu gracza i nie zależy od tego, czy gracz pokonał krupiera. W analizowanym przykładzie zakład Trips wygrywa wtedy, gdy gracz uzyska układ co najmniej trójki. Przykładową tabelę wypłat zakładu Trips przedstawiono w tabeli 2.5 [1].

Tabela 2.5. Przykładowa tabela wypłat zakładu Trips w Ultimate Texas Hold'em

Układ gracza	Wypłata
Poker królewski	50:1
Poker	40:1
Kareta	30:1
Full	8:1
Kolor	7:1
Strit	4:1
Trójka	3:1

W niektórych wariantach gry dostępny jest również zakład Progressive, którego tabela wypłat zależy od konkretnego kasyna oraz wartości puli progresywnej. Ze względu na zależność od zewnętrznych zasad kasynowych oraz brak bezpośredniego związku z podstawową decyzją Play, zakład ten nie będzie uwzględniany w podstawowym modelu środowiska.

2.5. Wartość oczekiwana, house edge i strategia bazowa

Wartość oczekiwana (ang. *expected value*, EV) określa średni wynik finansowy decyzji lub strategii przy założeniu dużej liczby powtórzeń gry. W kontekście Ultimate Texas Hold'em pojedyncze rozdanie może zakończyć się zarówno zyskiem, jak i stratą, jednak jakość strategii należy oceniać na podstawie średniego wyniku uzyskiwanego w wielu rozdaniach.

Przewaga kasyna (ang. *house edge*) jest matematyczną miarą określającą oczekiwany zysk kasyna względem zakładu gracza. Nie oznacza ona, że kasyno wygrywa każde

pojedyncze rozdanie, lecz że przy dużej liczbie rozegranych rozdań średni wynik będzie przesuwał się na korzyść kasyna. Gracz może więc osiągać krótkoterminowe zyski, jednak w długim okresie wynik gry powinien zbliżać się do wartości oczekiwanej wynikającej z zasad gry [4].

Istotne jest również to, że przewaga kasyna odnosi się do kwoty postawionych zakładów, a nie wyłącznie do początkowego kapitału gracza (ang. *bankroll*). Jeżeli gracz wykonuje wiele zakładów w kolejnych rozdaniach, całkowita kwota zaangażowana w grę może być wielokrotnie większa niż początkowy kapitał. Z tego powodu nawet niewielka procentowa przewaga kasyna może prowadzić do istotnej oczekiwanej straty w długim horyzoncie czasowym [4].

W grach takich jak Ultimate Texas Hold'em interpretacja house edge wymaga dodatkowej ostrożności, ponieważ gracz oprócz zakładów początkowych może zwiększać swoje zaangażowanie poprzez zakład Play. Z tego powodu oprócz klasycznej przewagi kasyna można analizować również miarę określaną jako element ryzyka (ang. *element of risk*), odnoszącą średnią stratę do całkowitej kwoty postawionej w rozdaniu [3], [5].

Przykładowe wartości przewagi kasyna dla wybranych gier przedstawiono w tabeli 2.6 [5].

Tabela 2.6. Przykładowe wartości przewagi kasyna dla wybranych gier kasynowych

Gra	Przewaga kasyna
Blackjack	0,28%
Ultimate Texas Hold'em	2,19%
Ruletka europejska	2,70%
Ruletka amerykańska	5,26%
Slot machines	2–15%

Tabela 2.6 pokazuje, że przewaga kasyna jest cechą konstrukcyjną gier hazardowych, ale jej poziom istotnie różni się pomiędzy grami. Celem niniejszej pracy nie jest wykazanie, że agent może trwale uzyskać dodatnią wartość oczekiwaną w grze zaprojektowanej na korzyść kasyna, lecz sprawdzenie, które metody uczenia przez wzmacnianie są w stanie nauczyć się strategii minimalizującej tę przewagę.

2.6. Uczenie przez wzmacnianie

Uczenie przez wzmacnianie (ang. *reinforcement learning*, RL) jest paradygmatem uczenia maszynowego, w którym agent uczy się podejmowania decyzji poprzez interakcję ze środowiskiem. W każdym kroku agent obserwuje aktualny stan (ang. *state*) środowiska, wybiera akcję (ang. *action*), a następnie otrzymuje nagrodę (ang. *reward*) oraz przechodzi do kolejnego stanu. Celem agenta jest wyuczenie strategii, określanej również jako polityka decyzyjna (ang. *policy*), która maksymalizuje sumę nagród w długim horyzoncie czasowym.

W przypadku Ultimate Texas Hold'em środowiskiem jest symulator gry, stanem jest aktualna sytuacja w rozdaniu, akcją jest decyzja gracza, natomiast nagrodą może być końcowy wynik finansowy rozdania. Takie sformułowanie pozwala traktować grę jako problem sekwencyjnego podejmowania decyzji, w którym agent uczy się kompromisu pomiędzy ryzykiem, ilością dostępnej informacji oraz potencjalną wysokością wypłaty.

Z perspektywy uczenia przez wzmocnianie Ultimate Texas Hold'em jest interesującym środowiskiem, ponieważ decyzje gracza są sekwencyjne i podejmowane przy różnym poziomie dostępnej informacji. Przed flopem gracz zna jedynie swoje dwie karty, ale może wykonać najwyższy zakład Play. Po flopie posiada więcej informacji, lecz maksymalny zakład jest mniejszy. Po odkryciu wszystkich kart wspólnych zna już pełny board, ale może wykonać wyłącznie zakład 1x Ante albo spasować. Agent musi więc nauczyć się kompromisu pomiędzy ryzykiem, ilością informacji oraz potencjalną wypłatą.

3. Projekt i implementacja środowiska Ultimate Texas Hold'em

W niniejszym rozdziale przedstawiono projekt oraz implementację środowiska Ultimate Texas Hold'em wykorzystywanego w dalszej części pracy do uczenia i oceny agentów. Opisane wcześniej zasady gry zostały odwzorowane w postaci deterministycznego i testowalnego silnika, który zarządza przebiegiem rozdania, udostępnia agentowi legalne akcje oraz rozlicza wynik finansowy rozgrywki.

Szczególną uwagę poświęcono zachowaniu niepełnej informacji charakterystycznej dla pokera. Agent otrzymuje wyłącznie informacje, które byłyby dostępne rzeczywistemu graczowi, natomiast karty krupiera, karty spalone oraz kolejność pozostałych kart w talii pozostają elementami wewnętrznego stanu środowiska. Takie rozdzielenie pozwala ograniczyć ryzyko niezamierzonego ujawnienia informacji podczas uczenia modeli.

Implementację podzielono na niezależne moduły odpowiedzialne między innymi za reprezentację kart, ocenę układów, sterowanie przebiegiem rozdania, walidację akcji oraz rozliczanie zakładów. Silnik domenowy nie zależy bezpośrednio od biblioteki Gymnasium. Adapter zgodny z jej interfejsem zostanie utworzony jako osobna warstwa, co umożliwi późniejsze porównywanie różnych reprezentacji stanu i funkcji nagrody bez modyfikowania zasad gry.

3.1. Założenia projektowe środowiska

Podstawową jednostką interakcji ze środowiskiem jest jedno kompletne rozdanie. Rozdanie rozpoczyna się od utworzenia talii i przekazania graczowi oraz krupierowi po dwóch kart własnych, a kończy się spasowaniem gracza albo automatycznym rozstrzygnięciem układów. Z punktu widzenia uczenia przez wzmacnianie pojedyncze rozdanie stanowi jeden epizod.

Agent podejmuje decyzje wyłącznie w trzech etapach: przed flopem, po odkryciu flopa oraz po odkryciu wszystkich pięciu kart wspólnych. W zależności od etapu może czekać, wykonać zakład Play o odpowiednim mnożniku albo spasować. Po wykonaniu zakładu Play środowisko automatycznie odkrywa pozostałe karty, ocenia ręce gracza i krupiera oraz przechodzi do stanu końcowego. W ramach jednego rozdania zakład Play może zostać wykonany tylko raz.

Środowisko modeluje podstawowy wariant Ultimate Texas Hold'em obejmujący zakłady Ante, Blind oraz Play. Zakłady boczne Trips i Progressive zostały pominięte. Nie wpływają one na przestrzeń decyzji podstawowej gry, natomiast zmieniają rozkład wyników finansowych i utrudniają ocenę jakości strategii decyzyjnej agenta. Uzasadnienie tego ograniczenia przedstawiono szerzej w dalszej części rozdziału.

Wartości zakładów wyrażono w jednostkach względnych, gdzie jedna jednostka odpowiada wartości zakładu Ante. Zakłady Ante i Blind mają zatem wartość jednej jednostki, natomiast wartość zakładu Play zależy od momentu jego wykonania i może wynosić od jednej do czterech jednostek. Takie rozwiązanie uniezależnia środowisko od konkretnej waluty i nominalnej wysokości stawki, a także ułatwia porównywanie wyników pomiędzy eksperymentami.

Silnik zwraca szczegółowe rozliczenie poszczególnych zakładów oraz łączny wynik netto rozdania. W podstawowym wariancie środowiska przeznaczonym dla agentów uczących się wynik netto będzie wykorzystywany jako nagroda terminalna. W krokach pośrednich nagroda wynosi zero. Rozdzielenie mechanizmu rozliczania gry od funkcji nagrody umożliwia późniejsze badanie alternatywnych wariantów nagrody bez ingerencji w reguły Ultimate Texas Hold'em.

Istotnym założeniem była również powtarzalność eksperymentów. Standardowa talia może być tasowana przy użyciu zadanego ziarna generatora liczb pseudolosowych, dzięki czemu możliwe jest odtworzenie całej sekwencji rozdań. W testach jednostkowych stosowana jest dodatkowo talia deterministyczna, pozwalająca jednoznacznie określić kolejność dobieranych kart i zweryfikować konkretne przypadki przebiegu oraz rozliczenia rozdania.

3.2. Architektura rozwiązania

Środowisko Ultimate Texas Hold'em zaprojektowano jako zestaw niezależnych modułów domenowych. Każdy moduł odpowiada za jeden wyraźnie określony obszar, taki jak reprezentacja kart, ocena układów, sterowanie przebiegiem rozdania albo rozliczanie zakładów. Centralnym elementem rozwiązania jest klasa `UTHGame`, która koordynuje działanie pozostałych komponentów, ale nie implementuje bezpośrednio wszystkich reguł gry.

Przyjęty podział ogranicza zależności pomiędzy poszczególnymi częściami systemu. Model kart i evaluator układów mogą być rozwijane oraz testowane niezależnie od zasad Ultimate Texas Hold'em. Analogicznie mechanizm rozliczania zakładów nie odpowiada za odkrywanie kart ani za walidację dozwolonych akcji. Dzięki temu zmiana jednego elementu, na przykład reprezentacji obserwacji agenta, nie wymaga modyfikowania sposobu oceny układów lub obliczania wypłat.

Najniższą warstwę rozwiązania stanowią moduły `cards` i `hands`. Pierwszy z nich definiuje karty oraz talię, natomiast drugi odpowiada za ocenę i porównywanie układów pokerowych. Moduły te nie zawierają reguł charakterystycznych dla Ultimate Texas Hold'em i mogą zostać ponownie wykorzystane w innych odmianach pokera.

Warstwę domenową gry tworzy pakiet `uth`. Zawiera on modele stanu, reguły legalności akcji, mechanizm rozdawania kart, sposób rozliczania zakładów oraz logikę sterującą kolejnymi fazami rozdania. Klasa `UTHGame` pełni rolę koordynatora tej war-

stwy. Na podstawie aktualnego stanu i wybranej akcji wywołuje odpowiednie operacje, aktualizuje stan rozdania oraz zwraca rezultat kroku.

Agent nie komunikuje się bezpośrednio z talią, evaluatorem ani pełnym stanem wewnętrznym gry. Otrzymuje obiekt `UTHObservation`, a swoją decyzję przekazuje do metody `step`. Rezultatem wykonania akcji jest obiekt `StepResult`, zawierający kolejną obserwację oraz, w przypadku zakończenia rozdania, jego wynik i szczegółowe rozliczenie. Takie rozwiązanie tworzy jednoznaczną granicę pomiędzy logiką środowiska a implementacją agenta.

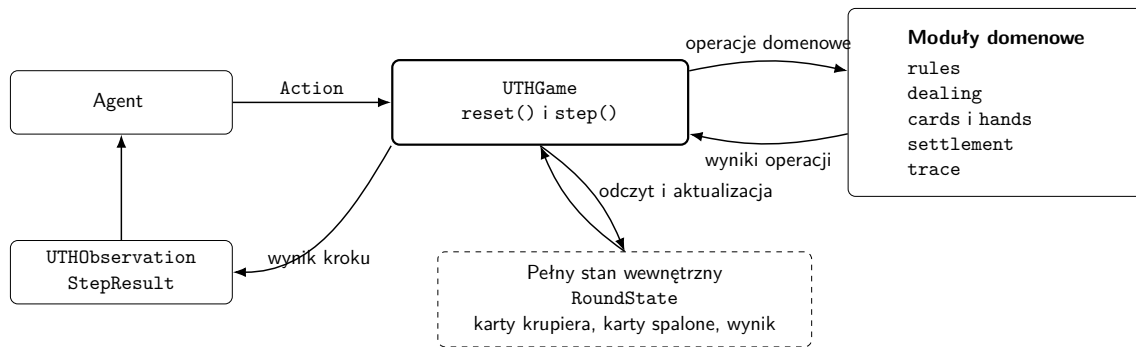
Tabela 3.1 przedstawia najważniejsze moduły rozwiązania i zakres ich odpowiedzialności.

Tabela 3.1. Podział odpowiedzialności pomiędzy modułami środowiska UTH

Moduł	Odpowiedzialność
<code>cards</code>	Reprezentacja rang, kolorów, pojedynczych kart oraz standardowej talii składającej się z 52 unikalnych kart.
<code>hands</code>	Ocena pięciokartowych układów, wybór najlepszego układu z pięciu, sześciu lub siedmiu kart oraz porównywanie wartości rąk.
<code>uth.models</code>	Definicja pełnego stanu rozdania, obserwacji agenta oraz wyniku pojedynczego kroku środowiska.
<code>uth.rules</code>	Określanie zbioru legalnych akcji dla każdej fazy rozdania.
<code>uth.dealing</code>	Realizacja kolejności rozdawania kart, odkrywania flopa, turna i rivera oraz obsługa kart spalonych.
<code>uth.card_source</code>	Abstrakcja źródła kart oraz deterministyczna talia wykorzystywana w testach.
<code>uth.settlement</code>	Określanie wyniku rozdania, kwalifikacji krupiera oraz rozliczanie zakładów Ante, Blind i Play.
<code>uth.game</code>	Koordinacja przebiegu rozdania, walidacja akcji, przechodzenie pomiędzy fazami oraz tworzenie wyników kolejnych kroków.
<code>uth.trace</code>	Opcjonalne rejestrowanie obserwacji, decyzji agenta i końcowego stanu rozdania na potrzeby diagnostyki.

Silnik domenowy nie zależy od konkretnej biblioteki uczenia przez wzmocnianie. Przyszły adapter Gymnasium będzie korzystał z publicznego interfejsu `UTHGame`, przekształcając obiekty domenowe na numeryczne reprezentacje wymagane przez algorytmy uczące się. Pozwala to zachować jedną implementację zasad gry dla wszystkich badanych reprezentacji obserwacji, funkcji nagrody i metod uczenia.

Ogólny przepływ informacji pomiędzy agentem, klasą sterującą i pozostałymi elementami silnika przedstawiono na rysunku 3.1. Agent komunikuje się wyłącznie z publicznym interfejsem klasy `UTHGame`. Pełny stan rozdania oraz moduły realizujące reguły gry pozostają ukryte za tą warstwą.



Rys. 3.1. Architektura i przepływ informacji w środowisku UTH

3.2.1. Przepływ danych podczas wykonania akcji

Po rozpoczęciu rozdania metoda `reset()` zwraca pierwszą obserwację agenta. Na jej podstawie agent wybiera akcję należącą do zbioru akcji legalnych w aktualnej fazie gry. Wybrana wartość jest następnie przekazywana do metody `step()`, która realizuje pojedyncze przejście środowiska.

W pierwszej kolejności silnik sprawdza, czy rozdanie nie zostało już zakończone, czy przekazana wartość jest poprawnym obiektem reprezentującym akcję oraz czy dana akcja jest legalna w aktualnej fazie. Odrzucenie niepoprawnej akcji następuje przed jakąkolwiek zmianą stanu lub pobraniem kart z talii. Zapobiega to częściowemu wykonaniu niedozwolonego przejścia.

Po pozytywnej walidacji sterowanie jest przekazywane do operacji właściwej dla aktualnej fazy: preflop, flopa albo rivera. Akcja oczekiwania powoduje odkrycie kart wymaganych do przejścia do następnej fazy. Wykonanie zakładu Play prowadzi natomiast do odkrycia wszystkich pozostałych kart wspólnych, oceny układów gracza i krupiera, określenia wyniku oraz rozliczenia aktywnych zakładów. Spasowanie na riverze kończy rozdanie bez wykonywania showdownu.

Po pomyślnym przeprowadzeniu operacji tworzony jest nowy obiekt `RoundState`. Na jego podstawie budowana jest obserwacja przeznaczona dla agenta oraz obiekt `StepResult`. Wynik kroku zawiera informację, czy rozdanie zostało zakończone, a w stanie terminalnym również rezultat rozgrywki, szczegółowe rozliczenie zakładów i dane dotyczące showdownu.

Jeżeli rejestrowanie historii jest włączone, obserwacja sprzed wykonania akcji i wybrana decyzja są zapisywane jako `DecisionRecord`. Rejestracja następuje dopiero po prawidłowym zakończeniu przejścia. Jeżeli podczas odkrywania kart lub aktualizacji stanu wystąpi wyjątek, decyzja nie jest dodawana do historii, a poprzedni stan środowiska pozostaje aktualny. Rozwiązanie to zapewnia spójność pomiędzy zapisanym przebiegiem rozdania a rzeczywiście wykonanymi przejściami.

3.3. Model kart i sterowanie talią

Reprezentacja kart i mechanizm sterowania talią stanowią podstawową warstwę środowiska. Elementy te zostały zaprojektowane niezależnie od zasad Ultimate Texas Hold'em, dzięki czemu mogą być wykorzystywane także przez inne warianty pokera oraz przez moduł oceniający układy.

3.3.1. Reprezentacja karty

Pojedyncza karta jest reprezentowana przez niemodyfikowalny obiekt `Card`, zawierający dwa pola: rangę oraz kolor. Rangi zapisano za pomocą typu wyliczeniowego `Rank`, natomiast kolory za pomocą typu `Suit`. Zastosowanie typów wyliczeniowych ogranicza możliwość utworzenia karty zawierającej niepoprawną wartość.

Rangi kart przyjmują wartości liczbowe od 2 do 14. Karty od dwójki do dziesiątki zachowują swoje naturalne wartości, natomiast walet, dama, król i as otrzymują odpowiednio wartości 11, 12, 13 i 14. Przyjęta reprezentacja umożliwia bezpośrednie porównywanie rang podczas oceny układów pokerowych. W podstawowej reprezentacji as jest kartą najwyższą, natomiast szczególny przypadek strita od asa do piątki obsługiwany jest przez evaluator układów.

Zbiór kolorów obejmuje trefle, karo, kiery i piki. Każdemu kolorowi oraz każdej randze przypisano również symbol tekstowy, umożliwiający czytelne przedstawienie kart, na przykład $A\spadesuit$ lub $10\heartsuit$. Reprezentacja tekstowa jest wykorzystywana głównie w testach, diagnostyce oraz zapisie przebiegu rozdania.

Obiekty kart są niemodyfikowalne po ich utworzeniu. Pozwala to bezpiecznie umieszczać je w krotkach i zbiorach, a także wykorzystywać porównanie liczności zbioru do wykrywania zduplikowanych kart. Konstruktor dodatkowo sprawdza, czy przekazana ranga i kolor należą do odpowiednich typów wyliczeniowych.

3.3.2. Standardowa talia

Standardową talię reprezentuje klasa `Deck`. Podczas jej tworzenia generowane są wszystkie kombinacje trzynastu rang i czterech kolorów. Liczba kart w pełnej talii wynosi zatem

$$|\mathcal{D}| = 13 \cdot 4 = 52. \quad (3.1)$$

Ponieważ każda kombinacja rangi i koloru występuje dokładnie raz, wygenerowana talia zawiera 52 unikalne karty. Karty są przechowywane wewnętrznie w modyfikowalnej kolekcji, jednak publiczny podgląd zawartości talii zwracany jest w postaci krotki. Unie umożliwia to zewnętrzną zmianę kolejności lub usunięcie karty z pominięciem operacji zdefiniowanych przez talię.

Podstawową operacją jest metoda `draw()`, która pobiera jedną kartę z wierzchu

talii i jednocześnie usuwa ją z kolekcji. Dostępna jest również operacja pobrania wielu kart. Próba pobrania karty z pustej talii lub większej liczby kart niż liczba pozostałych elementów powoduje zgłoszenie wyjątku. Takie zachowanie umożliwia szybkie wykrycie niepoprawnej implementacji przebiegu rozdania.

Domyślnie talia jest tasowana bezpośrednio po utworzeniu. Możliwe jest także utworzenie talii w kolejności początkowej, co bywa użyteczne podczas testowania samego modelu kart. Tasowanie wykonywane jest przy użyciu lokalnego generatora liczb pseudolosowych, a nie globalnego stanu losowości programu.

3.3.3. Abstrakcja źródła kart

Silnik rozdania nie zależy bezpośrednio od klasy `Deck`. Zamiast tego korzysta z abstrakcji `CardSource`, która wymaga jedynie udostępnienia operacji `draw()`. Każdy obiekt spełniający ten kontrakt może zostać użyty jako źródło kolejnych kart.

Minimalny interfejs źródła kart oddziela logikę rozdania od sposobu przechowywania i wybierania kart. W normalnej rozgrywce źródłem jest pseudolosowo przetasowana talia, natomiast w testach można przekazać źródło deterministyczne. Moduł odpowiedzialny za rozdawanie wykonuje w obu przypadkach te same operacje i nie musi rozpoznawać rodzaju używanej talii.

3.3.4. Deterministyczna talia testowa

Do testowania konkretnych scenariuszy wprowadzono klasę `FixedDeck`. Podczas jej tworzenia przekazywana jest jawna sekwencja kart określająca kolejność dobierania. Pierwszy element przekazanej sekwencji jest pierwszą kartą zwracaną przez metodę `draw()`.

Przed zapisaniem sekwencji sprawdzane jest, czy wszystkie jej elementy są poprawnymi obiektami `Card` oraz czy żadna karta nie występuje więcej niż raz. Dzięki temu błąd w danych testowych nie może doprowadzić do przeprowadzenia rozdania zawierającego dwie identyczne karty.

Deterministyczne źródło umożliwia przygotowanie testów dla ściśle określonych sytuacji, takich jak:

- wygrana gracza przy niekwalifikującym się krupierze,
- remis obu stron,
- uzyskanie konkretnego układu wypłacanego przez zakład `Blind`,
- sprawdzenie kolejności kart gracza i krupiera,
- weryfikacja kart spalonych przed flopem oraz przed turnem,
- próba wykonania przejścia przy niewystarczającej liczbie kart.

Porównanie dwóch dostępnych źródeł kart przedstawiono w tabeli 3.2.

Tabela 3.2. Porównanie źródeł kart wykorzystywanych przez środowisko

Cecha	Deck	FixedDeck
Zastosowanie	Standardowe rozdania i eksperymenty	Testy deterministyczne i odtwarzanie scenariuszy
Kolejność kart	Ustalana przez tasowanie	Jawnie przekazywana podczas tworzenia
Obsługa ziarna	Tak	Nie jest wymagana
Kontrola duplikatów	Wynika ze sposobu generowania pełnej talii	Walidowana podczas tworzenia obiektu
Pierwsza dobierana karta	Wynika z kolejności po tasowaniu	Pierwszy element zadanej sekwencji

3.3.5. Powtarzalność losowych rozdań

Powtarzalność eksperymentów zapewniono przez możliwość przekazania ziarna generatora liczb pseudolosowych. Utworzenie dwóch talii z tym samym ziarnem prowadzi do uzyskania tej samej kolejności kart, o ile wykonano na nich tę samą sekwencję operacji.

Na poziomie klasy `UTHGame` zastosowano dodatkowy generator nadrzędny. Przy każdym rozpoczęciu nowego rozdania generuje on osobne ziarno przekazywane do nowej talii. W rezultacie kolejne rozdania w obrębie jednego eksperymentu różnią się od siebie, ale cała ich sekwencja może zostać odtworzona przez ponowne uruchomienie środowiska z tym samym ziarnem początkowym.

Rozwiązanie to pozwala porównywać agentów na odpowiadających sobie sekwencjach rozdań. Jeżeli dwa eksperymenty zostaną uruchomione z tym samym ziarnem środowiska i zachowają tę samą kolejność wywołań, otrzymają identyczne strumienie talii. Ogranicza to wpływ losowego doboru kart na ocenę różnic pomiędzy badanymi strategiami.

3.4. Ocena układów pokerowych

Moduł oceny układów pokerowych został zaimplementowany niezależnie od reguł `Ultimate Texas Hold'em`. Jego zadaniem jest określenie wartości dowolnego poprawnego układu pięciokartowego oraz wybranie najlepszej możliwej ręki z pięciu, sześciu lub siedmiu dostępnych kart. W środowisku UTH evaluator jest wykorzystywany podczas showdownu osobno dla gracza i krupiera.

Hierarchię układów pokerowych oraz ich znaczenie przedstawiono w rozdziale teoretycznym. W niniejszej sekcji opisano sposób odwzorowania tej hierarchii w implementacji oraz mechanizm jednoznacznego porównywania dwóch rąk.

3.4.1. Reprezentacja wartości ręki

Kategorię układu reprezentuje typ wyliczeniowy `HandRank`. Kolejnym kategoriom przypisano rosnące wartości liczbowe, począwszy od wysokiej karty, a kończąc na pokerze. Dzięki temu porównanie kategorii może zostać wykonane przez porównanie odpowiadających im wartości całkowitych.

Sama kategoria nie wystarcza jednak do rozstrzygnięcia wszystkich przypadków. Dwie ręce mogą należeć do tej samej kategorii, lecz różnić się rangą kart tworzących układ albo kart dodatkowych. Z tego względu pełna wartość ręki jest reprezentowana przez obiekt `HandValue`, zawierający:

- kategorię układu `rank`,
- uporządkowaną krotkę wartości rozstrzygających remis `tiebreak`.

Klucz porównawczy ręki można zapisać jako

$$K(h) = (r(h), t_1(h), t_2(h), \dots, t_n(h)), \quad (3.2)$$

gdzie $r(h)$ oznacza wartość kategorii układu, natomiast $t_1(h), \dots, t_n(h)$ są kolejnymi wartościami rozstrzygającymi remis. Dwa klucze porównywane są leksykograficznie. W pierwszej kolejności porównywana jest kategoria układu, a dopiero w przypadku jej równości kolejne elementy krotki `tiebreak`.

Przykładowo wartość pary asów z królem, dziesiątką i dziewiątką jako kartami dodatkowymi może zostać zapisana w postaci

$$K(h) = (\text{ONE_PAIR}, 14, 13, 10, 9). \quad (3.3)$$

Jeżeli druga ręka również zawiera parę asów, porównanie przechodzi kolejno do króla, dziesiątki i dziewiątki. Kolory kart nie uczestniczą w rozstrzyganiu remisów, zgodnie ze standardowymi zasadami pokera.

Sposób budowy krotki rozstrzygającej dla każdej kategorii przedstawiono w tabeli 3.3.

Obiekt `HandValue` sprawdza zgodność długości krotki `tiebreak` z kategorią układu. Walidowane jest również, czy wszystkie jej elementy są liczbami całkowitymi odpowiadającymi poprawnym rangom kart. Ogranicza to możliwość utworzenia wartości ręki, która nie mogłaby powstać w prawidłowym rozdaniu.

3.4.2. Ocena układu pięciokartowego

Funkcja `evaluate_five_card_hand()` przyjmuje dokładnie pięć unikalnych kart. Przed rozpoczęciem oceny sprawdzana jest liczba kart, typ każdego elementu oraz brak duplikatów.

Tabela 3.3. Wartości wykorzystywane do rozstrzygania remisów

Kategoria	Liczba wartości	Kolejność elementów tiebreak
Wysoka karta	5	Wszystkie rangi w kolejności malejącej.
Jedna para	4	Ranga pary, a następnie trzy pozostałe rangi malejąco.
Dwie pary	3	Wyższa para, niższa para oraz karta dodatkowa.
Trójka	3	Ranga trójki oraz dwie pozostałe rangi malejąco.
Strit	1	Ranga najwyższej karty strita.
Kolor	5	Wszystkie rangi w kolejności malejącej.
Full	2	Ranga trójki, a następnie ranga pary.
Kareta	2	Ranga karety oraz ranga karty dodatkowej.
Poker	1	Ranga najwyższej karty pokera.

W pierwszym kroku obliczana jest liczba wystąpień każdej rangi. Na tej podstawie możliwe jest wykrycie par, dwóch par, trójki, fulla oraz karety. Niezależnie sprawdzane są dwa warunki:

- czy wszystkie karty mają ten sam kolor,
- czy pięć różnych rang tworzy ciąg kolejnych wartości.

Kategorie sprawdzane są od najsilniejszej do najsłabszej. Jeżeli spełnione są jednocześnie warunki strita i koloru, zwracany jest poker. W przeciwnym razie sprawdzane są kolejno kareta, full, kolor, strit, trójka, dwie pary, jedna para oraz wysoka karta.

Wartość tiebreak jest budowana bezpośrednio podczas rozpoznawania kategorii. Wynikiem funkcji nie jest zatem jedynie nazwa układu, lecz kompletna wartość umożliwiająca porównanie z dowolną inną poprawną ręką.

3.4.3. Obsługa niskiego strita z asem

W podstawowym modelu rang as otrzymuje wartość 14 i jest najwyższą kartą. W przypadku strita A-2-3-4-5 musi jednak pełnić funkcję karty niższej od dwójki. Układ ten jest obsługiwany jako szczególny przypadek.

Dla zestawu rang

$$\{14, 5, 4, 3, 2\} \quad (3.4)$$

evaluator zwraca strita o najwyższej karcie równej 5. Jego klucz porównawczy jest zatem słabszy od klucza strita zakończonego szóstką. Pozwala to zachować naturalną kolejność wszystkich możliwych stritów bez zmieniania podstawowej wartości asa w pozostałych kategoriach.

3.4.4. Wybór najlepszej ręki

Podczas showdownu gracz i krupier dysponują łącznie siedmioma kartami: dwiema kartami własnymi oraz pięcioma kartami wspólnymi. Rękę pokerową tworzy jednak dokładnie pięć kart. Funkcja `evaluate_best_hand()` generuje wszystkie możliwe pięciokartowe podzbiory dostępnych kart, ocenia każdy z nich, a następnie wybiera układ o największej wartości `HandValue`.

Liczba sprawdzanych kombinacji dla n dostępnych kart wynosi

$$N(n) = \binom{n}{5}. \quad (3.5)$$

Dla liczby kart obsługiwanych przez evaluator otrzymuje się:

$$\binom{5}{5} = 1, \quad \binom{6}{5} = 6, \quad \binom{7}{5} = 21. \quad (3.6)$$

W przypadku standardowego showdownu UTH konieczne jest zatem sprawdzenie 21 kombinacji dla gracza oraz 21 kombinacji dla krupiera. Ze względu na niewielką i stałą liczbę możliwości pełne przeszukanie jest wystarczająco proste, jednoznaczne oraz łatwe do zweryfikowania w testach. Nie było potrzeby stosowania bardziej złożonego, specjalizowanego algorytmu ewaluacji.

Przed wygenerowaniem kombinacji sprawdzane jest, czy przekazano od pięciu do siedmiu kart, czy wszystkie elementy są kartami oraz czy żadna karta nie występuje więcej niż raz.

Wynikiem operacji jest obiekt `EvaluatedHand`, który przechowuje zarówno wartość `HandValue`, jak i dokładnie pięć kart tworzących wybrany układ. Zachowanie kart składowych jest przydatne podczas:

- prezentowania wyniku showdownu,
- analizy decyzji agentów,
- diagnozowania błędów evaluatora,
- identyfikowania rzadkich lub szczególnie wartościowych rozdań.

Poker królewski nie stanowi osobnej kategorii `HandRank`. Jest reprezentowany jako poker z asem jako najwyższą kartą. Obiekt `HandValue` udostępnia właściwość pozwalającą rozpoznać ten przypadek, co jest potrzebne przy naliczaniu najwyższej wypłaty zakładu `Blind`.

3.5. Stan wewnętrzny i obserwacja agenta

Ultimate Texas Hold'em jest grą z niepełną informacją. W momencie podejmowania decyzji gracz zna własne karty oraz dotychczas odkryte karty wspólne, ale nie

zna kart krupiera, kart spalonych ani kolejności pozostałych kart w talii. Poprawne odwzorowanie tej własności wymaga rozdzielenia pełnego stanu środowiska od informacji udostępnianych agentowi.

W implementacji zastosowano dwa odrębne modele danych:

- `RoundState`, reprezentujący pełny stan wewnętrzny rozdania,
- `UTHObservation`, reprezentujący obserwację dostępną agentowi.

Agent nie otrzymuje bezpośredniego odwołania do obiektu `RoundState`. Każda obserwacja jest tworzona na jego podstawie przez osobną funkcję projekcji, która kopiuje wyłącznie informacje dozwolone z perspektywy gracza.

3.5.1. Pełny stan rozdania

Obiekt `RoundState` zawiera wszystkie dane potrzebne do kontynuowania i rozstrzygnięcia rozdania. Obejmuje on:

- aktualną fazę gry,
- dwie karty własne gracza,
- dwie ukryte karty krupiera,
- odkryte karty wspólne,
- karty spalone,
- mnożnik wykonanego zakładu `Play`,
- końcowy wynik rozdania,
- szczegółowe rozliczenie zakładów,
- rezultat showdownu.

Nie wszystkie pola są aktywne jednocześnie. W stanach pośrednich wynik rozdania, rozliczenie, rezultat showdownu oraz mnożnik zakładu `Play` pozostają nieokreślone. Pola te otrzymują wartości dopiero po przejściu do stanu terminalnego.

Liczba kart wspólnych i spalonych jest uzależniona od aktualnej fazy. Zależność tę przedstawiono w tabeli 3.4.

Obiekt stanu jest niemodyfikowalny. Każde przejście środowiska prowadzi do utworzenia nowej instancji `RoundState`, zamiast modyfikowania istniejącego obiektu. Upraszcza to analizę przejść, ogranicza ryzyko częściowej aktualizacji oraz ułatwia tworzenie deterministycznych testów.

Podczas tworzenia stanu sprawdzane są między innymi:

Tabela 3.4. Liczba kart przechowywanych w stanie w poszczególnych fazach

Faza	Karty wspólne	Karty spalone	Znaczenie
PREFLOP	0	0	Rozdano wyłącznie karty własne gracza i krupiera.
FLOP	3	1	Spalono jedną kartę i odkryto flop.
RIVER	5	2	Odkryto komplet kart wspólnych.
TERMINAL	5	2	Rozdanie zostało zakończone foldem albo showdownem.

- poprawny typ fazy,
- obecność dokładnie dwóch kart gracza i dwóch kart krupiera,
- liczba kart wspólnych właściwa dla danej fazy,
- liczba kart spalonych właściwa dla danej fazy,
- brak zduplikowanych kart w całym stanie,
- zgodność pól końcowych z fazą rozdania.

Walidacja obejmuje również zależności semantyczne pomiędzy polami. Stan niekońcowy nie może zawierać wyniku ani rozliczenia. Stan zakończony foldem nie może zawierać mnożnika Play ani rezultatu showdownu. Z kolei stan zakończony showdownem musi zawierać zarówno mnożnik wykonanego zakładu, jak i szczegóły ocenionych rąk.

3.5.2. Obserwacja agenta

Obiekt `UTHObservation` zawiera wyłącznie dane, które mogą zostać wykorzystane przez agenta podczas podejmowania decyzji. Są to:

- aktualna faza gry,
- dwie karty własne gracza,
- dotychczas odkryte karty wspólne,
- zbiór legalnych akcji,
- mnożnik zakładu Play w obserwacji terminalnej.

Obserwacja celowo nie zawiera:

- kart krupiera,

- kart spalonych,
- pozostałych kart w talii,
- kolejności przyszłego dobierania kart,
- wyniku ewaluacji ręki krupiera przed zakończeniem rozdania.

Zbiór legalnych akcji jest umieszczony bezpośrednio w obserwacji. Agent nie musi zatem samodzielnie odtwarzać zasad gry na podstawie fazy. Rozwiązanie to ułatwia implementację agentów bazowych, a w przyszłym adapterze Gymnasium umożliwi utworzenie maski akcji.

Obserwacja, podobnie jak pełny stan, jest obiektem niemodyfikowalnym. Sprawdzana jest liczba widocznych kart, brak duplikatów oraz zgodność zbioru legalnych akcji z aktualną fazą gry. Dzięki temu błędnie skonstruowana obserwacja jest odrzucana przed przekazaniem jej agentowi.

3.5.3. Projekcja stanu na obserwację

Proces tworzenia obserwacji można przedstawić jako funkcję projekcji

$$O_t = \phi(S_t), \quad (3.7)$$

gdzie S_t jest pełnym stanem środowiska w chwili t , natomiast O_t oznacza obserwację udostępnioną agentowi.

Dla zaimplementowanego środowiska projekcja ma postać

$$\phi(S_t) = (p_t, C_t^{player}, C_t^{community}, A_t^{legal}, m_t), \quad (3.8)$$

gdzie:

- p_t oznacza aktualną fazę gry,
- C_t^{player} oznacza karty własne gracza,
- $C_t^{community}$ oznacza odkryte karty wspólne,
- A_t^{legal} oznacza zbiór legalnych akcji,
- m_t oznacza mnożnik zakładu Play, jeżeli został już wykonany.

Elementy takie jak karty krupiera, karty spalone i zawartość pozostałej talii należą do S_t , ale nie są elementami O_t .

Porównanie zawartości pełnego stanu i obserwacji przedstawiono w tabeli 3.5.

Wynik rozdania, rozliczenie i szczegóły showdownu są po zakończeniu epizodu zwracane przez obiekt `StepResult`, a nie przez samą obserwację. Informacje te pojawiają

Tabela 3.5. Porównanie pełnego stanu środowiska i obserwacji agenta

Informacja	RoundState	UTHObservation
Aktualna faza	tak	tak
Karty własne gracza	tak	tak
Odkryte karty wspólne	tak	tak
Karty krupiera	tak	nie
Karty spalone	tak	nie
Kolejność pozostałej talii	dostępna wewnętrznie silnikowi	nie
Legalne akcje	wyznaczane na podstawie fazy	tak
Mnożnik zakładu Play	tak	tylko po wykonaniu zakładu
Końcowy wynik rozdania	tak	nie
Szczegółowe rozliczenie	tak	nie
Szczegóły showdownu	tak	nie

się dopiero po wykonaniu ostatniej decyzji, dlatego nie mogą wpłynąć na wybór akcji w bieżącym rozdaniu.

3.5.4. Ochrona przed wyciekami informacji

Rozdzielenie stanu i obserwacji pełni rolę ochrony przed wyciekami informacji ukrytej. Funkcja tworząca obserwację nie usuwa niepożądanych pól z kopii pełnego stanu. Zamiast tego jawnie konstruuje nowy obiekt, przekazując wyłącznie dozwolone wartości.

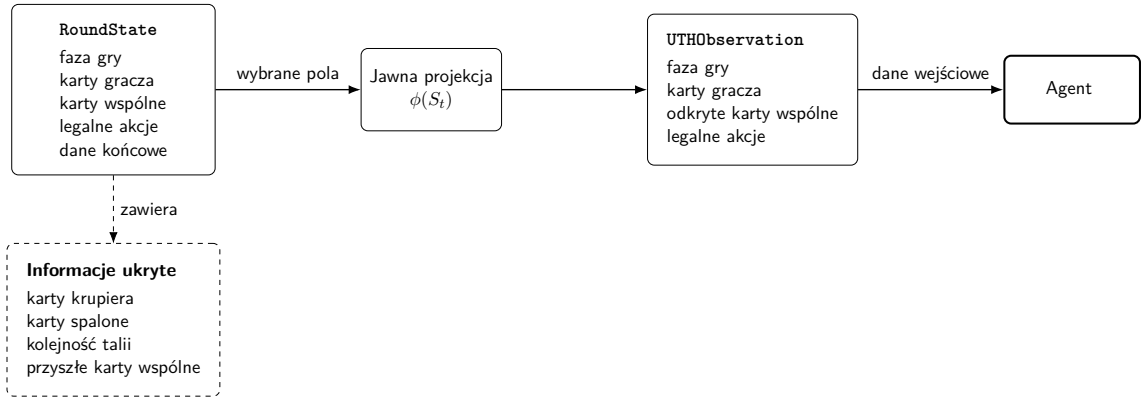
Podjęcie to można określić jako tworzenie obserwacji zgodnie z listą dozwolonych informacji. Jest ono bezpieczniejsze niż stosowanie listy pól zabronionych. Po dodaniu nowego pola do RoundState nie zostanie ono automatycznie ujawnione agentowi. Aby znalazło się w obserwacji, musi zostać jawnie dodane do modelu UTHObservation oraz do funkcji projekcji.

Istotne jest również rozróżnienie pomiędzy interfejsem przeznaczonym dla agenta a interfejsem diagnostycznym. Pełny stan może być wykorzystywany przez testy, rejestrowanie przebiegu rozdania i analizę zakończonych epizodów, ale nie powinien być przekazywany metodzie wybierającej akcję agenta.

Granica pomiędzy pełnym stanem środowiska a informacjami przekazywanymi agentowi została przedstawiona na rysunku 3.2. Obserwacja nie jest bezpośrednim widokiem pełnego stanu, lecz wynikiem jawnie zdefiniowanej projekcji. Informacje ukryte pozostają dostępne wyłącznie wewnętrznym komponentom środowiska.

Informacje ukryte są częścią stanu potrzebnego do poprawnego prowadzenia gry, lecz nie uczestniczą w tworzeniu obserwacji. Przykładowo silnik musi przechowywać karty krupiera, aby po zakończeniu rozdania przeprowadzić showdown, ale agent nie może korzystać z nich podczas wyboru akcji.

Podobnie karty spalone i kolejność pozostałych kart są potrzebne do kontynuowania rozdania oraz zapewnienia jego deterministycznego odtworzenia, jednak ich ujawnie-



Rys. 3.2. Oddzielenie pełnego stanu środowiska od obserwacji agenta

nie zmieniałoby problem decyzyjny. Agent znałby wówczas przyszłe zdarzenia losowe, co prowadziłoby do sztucznie zawyżonej jakości strategii.

Jawna funkcja projekcji stanowi zatem część definicji środowiska z niepełną informacją. Zapewnia ona, że każda metoda wyboru akcji, niezależnie od zastosowanego algorytmu, otrzymuje ten sam zakres informacji. W przyszłych eksperymentach zmianie może podlegać sposób numerycznego kodowania obserwacji, natomiast zbiór informacji dostępnych agentowi pozostanie niezmienny.

3.6. Przebieg rozdania i maszyna stanów

Przebieg pojedynczego rozdania Ultimate Texas Hold'em odwzorowano jako deterministyczną maszynę stanów. Aktualna faza określa zakres informacji widocznych dla agenta, zbiór legalnych akcji oraz operacje wykonywane przez środowisko po otrzymaniu decyzji.

Zbiór faz środowiska ma postać

$$\mathcal{P} = \{\text{PREFLOP}, \text{FLOP}, \text{RIVER}, \text{TERMINAL}\}. \quad (3.9)$$

Każde poprawne wykonanie metody `step()` prowadzi do przejścia do następnej fazy albo do zakończenia rozdania. Agent nie może pozostać w tej samej fazie po wykonaniu akcji.

3.6.1. Rozpoczęcie rozdania

Nowe rozdanie rozpoczynane jest przez metodę `reset()`. Jeżeli nie przekazano zewnętrznego źródła kart, środowisko tworzy nową przetasowaną talię. Następnie dobierane są cztery karty własne w kolejności

$$P_1 \rightarrow D_1 \rightarrow P_2 \rightarrow D_2, \quad (3.10)$$

gdzie P_i oznacza kartę gracza, a D_i kartę krupiera. Utworzony stan znajduje się

w fazie PREFLOP. Agent widzi swoje dwie karty, lecz karty krupiera pozostają ukryte.

Na tym etapie nie są jeszcze dobierane karty wspólne ani karty spalone. Metoda `reset()` zwraca pierwszą obserwację, na podstawie której agent podejmuje decyzję przed flopem.

3.6.2. Faza preflop

W fazie PREFLOP dostępne są trzy akcje:

- CHECK,
- BET_3X,
- BET_4X.

Akcja CHECK oznacza odłożenie decyzji o zakładzie Play. Środowisko spala jedną kartę, odkrywa trzy karty flopa i przechodzi do fazy FLOP.

Wykonanie BET_3X albo BET_4X ustala wartość zakładu Play odpowiednio na trzy lub cztery jednostki Ante. Po postawieniu zakładu agent nie podejmuje już kolejnych decyzji. Środowisko odkrywa wszystkie pozostałe karty wspólne, przeprowadza showdown, rozlicza zakłady i przechodzi bezpośrednio do fazy TERMINAL.

3.6.3. Faza flopa

Przejście do fazy FLOP następuje wyłącznie po wykonaniu akcji CHECK przed flopem. Stan zawiera wówczas trzy odkryte karty wspólne i jedną kartę spaloną.

Agent może wykonać jedną z dwóch akcji:

- CHECK,
- BET_2X.

Ponowne wykonanie CHECK powoduje spalenie kolejnej karty oraz jednoczesne odkrycie turna i rivera. Środowisko przechodzi następnie do fazy RIVER.

Akcja BET_2X ustala zakład Play na dwie jednostki Ante. Środowisko odkrywa turn i river, przeprowadza showdown i kończy rozdanie.

3.6.4. Faza rivera

W fazie RIVER wszystkie pięć kart wspólnych jest już odkrytych. Nazwa fazy oznacza ostatni etap decyzyjny, a nie samo odkrycie pojedynczej karty rivera. Turn i river są odkrywane razem, ponieważ pomiędzy nimi gracz nie podejmuje żadnej decyzji.

W tej fazie agent musi wybrać jedną z dwóch akcji:

- BET_1X,

- FOLD.

Akcja BET_1X ustala zakład Play na jedną jednostkę Ante i prowadzi do showdownu. Akcja FOLD kończy rozdanie bez porównywania układów. W obu przypadkach kolejną fazą jest TERMINAL.

3.6.5. Faza terminalna

Faza TERMINAL oznacza zakończenie epizodu. Stan zawiera końcowy wynik rozdania oraz rozliczenie zakładów. W przypadku zakończenia przez zakład Play zawiera również rezultat showdownu i wartość mnożnika Play.

Stan terminalny nie udostępnia żadnych legalnych akcji. Próba ponownego wywołania metody `step()` jest traktowana jako błąd domenowy. Rozpoczęcie kolejnego epizodu wymaga ponownego wywołania metody `reset()`.

3.6.6. Tabela przejść

Pełny zbiór przejść pomiędzy fazami przedstawiono w tabeli 3.6.

Tabela 3.6. Przejścia pomiędzy fazami środowiska UTH

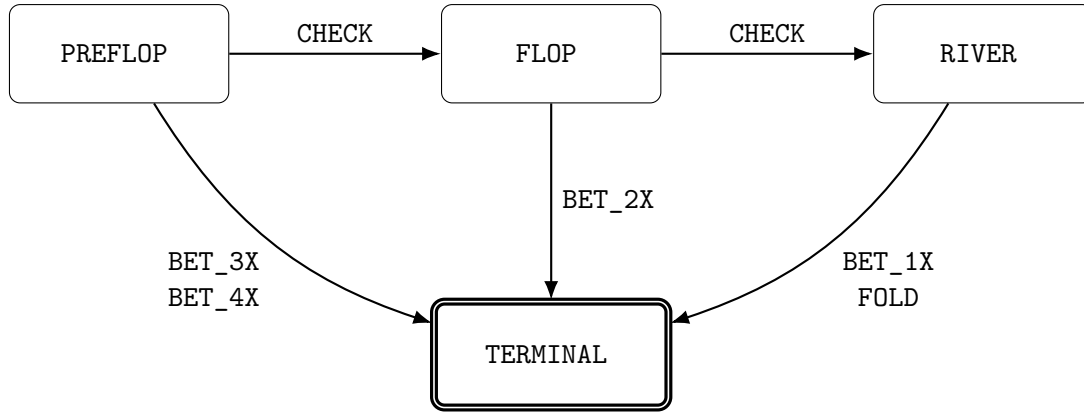
Faza	Akcja	Następna faza	Operacje środowiska
PREFLOP	CHECK	FLOP	Spalenie jednej karty i odkrycie trzech kart flopa.
PREFLOP	BET_3X	TERMINAL	Odkrycie całego stołu, showdown i rozliczenie Play 3×
PREFLOP	BET_4X	TERMINAL	Odkrycie całego stołu, showdown i rozliczenie Play 4×
FLOP	CHECK	RIVER	Spalenie jednej karty oraz odkrycie turna i rivera.
FLOP	BET_2X	TERMINAL	Odkrycie turna i rivera, showdown i rozliczenie Play 2×
RIVER	BET_1X	TERMINAL	Showdown i rozliczenie Play 1×
RIVER	FOLD	TERMINAL	Zakończenie bez showdownu i rozliczenie spasowania.

3.6.7. Diagram maszyny stanów

Graficzną reprezentację przejść przedstawiono na rysunku 3.3.

3.6.8. Zagwarantowanie pojedynczego zakładu Play

Struktura maszyny stanów gwarantuje, że zakład Play może zostać wykonany najwyżej raz. Każda akcja typu BET prowadzi bezpośrednio do fazy terminalnej. Nie istnieje więc przejście, w którym agent po postawieniu Play otrzymałby możliwość wykonania kolejnego zakładu.



Rys. 3.3. Maszyna stanów pojedynczego rozdania UTH

Akcje CHECK nie tworzą zakładu, lecz przesuwają decyzję do późniejszej fazy. Maksymalna liczba decyzji w jednym rozdaniu wynosi zatem trzy i występuje w ścieżce

$$\text{PREFLOP} \xrightarrow{\text{CHECK}} \text{FLOP} \xrightarrow{\text{CHECK}} \text{RIVER} \xrightarrow{\text{BET_1X lub FOLD}} \text{TERMINAL}. \quad (3.11)$$

Najkrótsza ścieżka zawiera jedną decyzję i występuje po wykonaniu zakładu $3\times$ lub $4\times$ przed flopem.

Przed wykonaniem przejścia środowisko sprawdza, czy przekazana akcja należy do zbioru legalnego dla bieżącej fazy. Operacje odkrywania kart dodatkowo sprawdzają oczekiwaną fazę wejściową. Ogranicza to możliwość pominięcia etapu, ponownego odkrycia tych samych kart lub wykonania akcji po zakończeniu rozdania.

3.7. Przestrzeń akcji i walidacja decyzji

Pełna przestrzeń akcji środowiska Ultimate Texas Hold'em składa się z sześciu decyzji domenowych:

$$\mathcal{A} = \{\text{CHECK}, \text{BET_4X}, \text{BET_3X}, \text{BET_2X}, \text{BET_1X}, \text{FOLD}\}. \quad (3.12)$$

Nie wszystkie elementy zbioru $\mathcal{A}$ są dostępne jednocześnie. Zbiór akcji legalnych zależy od aktualnej fazy rozdania. Dla fazy p zdefiniowano funkcję

$$\mathcal{A}_{\text{legal}}(p) \subseteq \mathcal{A}, \quad (3.13)$$

która zwraca akcje dozwolone w danym momencie rozgrywki.

Zależność pomiędzy fazą i zbiorem legalnych akcji można zapisać następująco:

$$\mathcal{A}_{legal}(p) = \begin{cases} \{\text{CHECK}, \text{BET_3X}, \text{BET_4X}\}, & p = \text{PREFLOP}, \\ \{\text{CHECK}, \text{BET_2X}\}, & p = \text{FLOP}, \\ \{\text{BET_1X}, \text{FOLD}\}, & p = \text{RIVER}, \\ \emptyset, & p = \text{TERMINAL}. \end{cases} \quad (3.14)$$

Znaczenie poszczególnych akcji przedstawiono w tabeli 3.7.

Tabela 3.7. Akcje dostępne w środowisku UTH

Akcja	Legalna faza	Znaczenie
CHECK	PREFLOP, FLOP	Odłożenie decyzji o zakładzie Play i przejście do kolejnego etapu rozdania.
BET_4X	PREFLOP	Postawienie zakładu Play o wartości czterech jednostek Ante i zakończenie części decyzyjnej rozdania.
BET_3X	PREFLOP	Postawienie zakładu Play o wartości trzech jednostek Ante i zakończenie części decyzyjnej rozdania.
BET_2X	FLOP	Postawienie zakładu Play o wartości dwóch jednostek Ante po odkryciu flopa.
BET_1X	RIVER	Postawienie zakładu Play o wartości jednej jednostki Ante po odkryciu wszystkich kart wspólnych.
FOLD	RIVER	Rezygnacja z dalszej gry i zakończenie rozdania bez showdownu.

3.7.1. Akcje legalne jako część obserwacji

Zbiór legalnych akcji jest wyznaczany przez środowisko na podstawie aktualnej fazy i umieszczany w obiekcie UTHObservation. Agent nie musi zatem samodzielnie odtwarzać reguł legalności na podstawie liczby kart wspólnych lub historii poprzednich decyzji.

Takie rozwiązanie ogranicza odpowiedzialność agenta do wyboru jednej akcji spośród wartości udostępnionych przez środowisko. Jest to szczególnie przydatne dla agentów bazowych, które mogą bezpośrednio losować lub wybierać element ze zbioru `legal_actions`.

Przechowywanie legalnych akcji w obserwacji nie oznacza, że środowisko ufa decyzji zwróconej przez agenta. Każda akcja jest ponownie walidowana przez metodę `step()`. Dzięki temu niepoprawnie zaimplementowany agent nie może wymusić przejścia sprzecznego z regułami gry.

3.7.2. Walidacja decyzji

Przed wykonaniem operacji zmieniającej stan środowisko przeprowadza kolejne etapy walidacji:

1. sprawdza, czy rozdanie zostało rozpoczęte,
2. sprawdza, czy aktualny stan nie jest terminalny,
3. sprawdza, czy przekazana wartość jest obiektem typu `Action`,
4. wyznacza zbiór akcji legalnych dla aktualnej fazy,
5. sprawdza, czy przekazana akcja należy do tego zbioru.

Jeżeli rozdanie nie zostało rozpoczęte, zgłaszany jest błąd `RoundNotStartedError`. Próba wykonania decyzji po zakończeniu epizodu prowadzi do zgłoszenia `RoundFinishedError`. Akcja należąca do typu `Action`, lecz niedozwolona w aktualnej fazie, powoduje zgłoszenie `IllegalActionError`.

Przykładem nielegalnej decyzji jest wykonanie `BET_2X` przed flopem albo próba spasowania w fazie `FLOP`. Środowisko nie zastępuje takiej akcji inną decyzją i nie nakłada automatycznej kary. Przejście jest odrzucane, a błąd musi zostać obsłużony przez komponent sterujący eksperymentem.

Walidacja następuje przed dobieraniem nowych kart i przed zapisaniem nowego stanu. Oznacza to, że nielegalna decyzja nie zużywa kart z talii, nie zmienia fazy rozdania i nie jest dodawana do historii decyzji.

3.7.3. Niezmienniki przestrzeni akcji

Wprowadzone reguły pozwalają sformułować następujące niezmienniki:

- `CHECK` jest dostępne wyłącznie przed riverem,
- `FOLD` jest dostępne wyłącznie po odkryciu wszystkich kart wspólnych,
- mnożnik zakładu `Play` jest jednoznacznie związany z fazą,
- każda akcja typu `BET` prowadzi do stanu terminalnego,
- po postawieniu zakładu `Play` nie można wykonać kolejnej decyzji,
- stan terminalny nie udostępnia żadnej akcji.

Wartość mnożnika nie jest przekazywana jako dowolny parametr liczbowy agenta. Zamiast jednej ogólnej akcji `BET` zastosowano osobne wartości `BET_1X`, `BET_2X`, `BET_3X` i `BET_4X`. Ogranicza to możliwość przekazania niepoprawnej wysokości zakładu, na przykład $5 \times$ Ante, oraz upraszcza późniejsze odwzorowanie akcji na przestrzeń dyskretną.

3.7.4. Planowane odwzorowanie na przestrzeń Gymnasium

Obecny silnik posługuje się wartościami domenowymi typu Action. Biblioteki uczenia przez wzmocnianie zwykle oczekują natomiast akcji reprezentowanych przez kolejne liczby całkowite. Przyszły adapter Gymnasium będzie zatem definiował stałe, jednoznaczne odwzorowanie

$$f : \mathcal{A} \rightarrow \{0, 1, \dots, |\mathcal{A}| - 1\}. \quad (3.15)$$

Ponieważ pełna przestrzeń zawiera sześć akcji, adapter będzie mógł wykorzystywać dyskretną przestrzeń o liczności sześciu. Dokładne przypisanie identyfikatorów liczbowych zostanie zdefiniowane w implementacji adaptera, aby dokumentacja odpowiadała rzeczywiście używanemu interfejsowi.

Stała przestrzeń dyskretna będzie zawierała również akcje niedostępne w niektórych fazach. Z tego względu obserwacja adaptera powinna udostępniać maskę legalności. Dla akcji a_i maskę można zdefiniować jako

$$M_i(s) = \begin{cases} 1, & a_i \in \mathcal{A}_{legal}(p(s)), \\ 0, & a_i \notin \mathcal{A}_{legal}(p(s)), \end{cases} \quad (3.16)$$

gdzie $p(s)$ oznacza fazę odpowiadającą stanowi s .

Maska pozwoli agentowi ograniczyć wybór do akcji zgodnych z regułami gry, nie zmieniając stałego rozmiaru przestrzeni wyjściowej modelu. Niezależnie od zastosowania maski adapter nadal będzie przekazywał decyzję do właściwego silnika, który zachowa końcową odpowiedzialność za walidację legalności.

3.8. Showdown i rozliczanie zakładów

Każde rozdanie, w którym gracz wykonał zakład Play, kończy się showdownem. Środowisko odkrywa wszystkie brakujące karty wspólne, ocenia najlepszą pięciokartową rękę gracza i krupiera, a następnie porównuje otrzymane wartości. Wynik porównania stanowi podstawę do rozliczenia zakładów Ante, Blind oraz Play.

Spasowanie gracza jest obsługiwane oddzielnie. W takim przypadku showdown nie jest przeprowadzany, ponieważ wynik rozdania wynika bezpośrednio z decyzji gracza.

3.8.1. Przebieg showdownu

Podczas showdownu każda strona dysponuje siedmioma kartami:

$$C^{player} = C_{private}^{player} \cup C^{community}, \quad (3.17)$$

$$C^{dealer} = C_{private}^{dealer} \cup C^{community}, \quad (3.18)$$

gdzie $C_{private}^{player}$ i $C_{private}^{dealer}$ oznaczają po dwie karty własne, natomiast $C^{community}$ zawiera pięć kart wspólnych.

Ewaluator wybiera najlepszy pięciokartowy układ osobno dla gracza i krupiera. Wynik tej operacji jest przechowywany w obiekcie `ShowdownResult`, który zawiera:

- najlepszą rękę gracza,
- najlepszą rękę krupiera,
- wynik porównania z perspektywy gracza,
- informację o kwalifikacji krupiera.

Wynik showdownu należy do zbioru

$$\mathcal{O}_{showdown} = \{\text{PLAYER_WIN}, \text{DEALER_WIN}, \text{PUSH}\}. \quad (3.19)$$

Dla wartości rąk H_p oraz H_d wynik można zdefiniować jako

$$O(H_p, H_d) = \begin{cases} \text{PLAYER_WIN}, & H_p > H_d, \\ \text{DEALER_WIN}, & H_p < H_d, \\ \text{PUSH}, & H_p = H_d. \end{cases} \quad (3.20)$$

Porównanie rąk jest wykonywane niezależnie od kwalifikacji krupiera. Oznacza to, że krupier może wygrać rozdanie również wtedy, gdy jego ręka nie spełnia warunku kwalifikacji. Przykładem jest sytuacja, w której obie strony mają wyłącznie wysoką kartę, ale wysoka karta krupiera jest silniejsza.

3.8.2. Kwalifikacja krupiera

W podstawowym wariancie Ultimate Texas Hold'em krupier kwalifikuje się, jeżeli posiada co najmniej jedną parę. Warunek kwalifikacji można zapisać jako

$$Q(H_d) = \begin{cases} 1, & \text{rank}(H_d) \geq \text{ONE_PAIR}, \\ 0, & \text{rank}(H_d) = \text{HIGH_CARD}. \end{cases} \quad (3.21)$$

Kwalifikacja nie zmienia wyniku porównania rąk. Wpływa natomiast na sposób rozliczenia zakładu Ante:

- jeżeli krupier kwalifikuje się, Ante wygrywa lub przegrywa zgodnie z wynikiem showdownu,
- jeżeli krupier nie kwalifikuje się, Ante jest zwracane niezależnie od tego, która strona ma silniejszą rękę.

Zakład Play jest zawsze rozliczany na podstawie bezpośredniego porównania rąk. Zakład Blind jest rozliczany według układu gracza w przypadku jego zwycięstwa, zwracany przy remisie oraz przegrywany przy zwycięstwie krupiera.

Zależności te przedstawiono w tabeli 3.8.

Tabela 3.8. Rozliczenie zakładów po showdownie

Wynik	Kwalifikacja	Ante	Blind	Play
Wygrana gracza	tak	wygrana 1:1	według tabeli wypłat	wygrana 1:1
Wygrana gracza	nie	zwrot	według tabeli wypłat	wygrana 1:1
Remis	dowolna	zwrot	zwrot	zwrot
Wygrana krupiera	tak	przegrana	przegrana	przegrana
Wygrana krupiera	nie	zwrot	przegrana	przegrana

Ostatni przypadek jest istotny z punktu widzenia poprawności implementacji. Brak kwalifikacji krupiera nie oznacza automatycznego zwycięstwa gracza. Jeżeli krupier ma silniejszą wysoką kartę, wygrywa porównanie, zakłady Blind i Play są przegrywane, natomiast Ante zostaje zwrócone.

3.8.3. Model rozliczenia zakładów

Wynik finansowy każdego zakładu reprezentuje obiekt `WagerSettlement`. Zawiera on dwie wartości:

- `stake` – początkową wartość postawionego zakładu,
- `net_profit` – zysk albo stratę netto.

Wszystkie kwoty są wyrażane w jednostkach Ante. Dla pojedynczego zakładu o stawce s i zysku netto n całkowity zwrot brutto wynosi

$$g = s + n. \quad (3.22)$$

Przykładowe wartości przedstawiono w tabeli 3.9.

Pełne rozliczenie rozdania jest reprezentowane przez obiekt `Settlement`, który zawiera osobne wyniki dla Ante, Blind oraz Play. Dzięki temu możliwa jest analiza zarówno łącznego wyniku rozdania, jak i wpływu poszczególnych składników.

Dla rozliczenia S całkowita postawiona kwota wynosi

$$S_{stake} = s_{Ante} + s_{Blind} + s_{Play}. \quad (3.23)$$

Łączny wynik netto jest równy

Tabela 3.9. Interpretacja wartości pojedynczego rozliczenia

Stawka	Zysk netto	Zwrot brutto	Znaczenie
1	1	2	Wygrana zakładu wypłacanego 1:1.
1	0	1	Zwrot początkowej stawki.
1	-1	0	Przegrana pełnej stawki.
0	0	0	Zakład nie został postawiony.

$$S_{net} = n_{Ante} + n_{Blind} + n_{Play}, \quad (3.24)$$

natomiast całkowity zwrot brutto wynosi

$$S_{gross} = g_{Ante} + g_{Blind} + g_{Play}. \quad (3.25)$$

Z definicji zachodzi zależność

$$S_{gross} = S_{stake} + S_{net}. \quad (3.26)$$

Wartości finansowe są reprezentowane za pomocą typu `Fraction`. Pozwala to przechowywać wypłaty takie jak $3/2$ bez stosowania przybliżeń zmiennoprzecinkowych. Ma to znaczenie podczas wielokrotnego sumowania wyników w eksperymentach, ponieważ eliminuje błędy zaokrągleń na poziomie pojedynczych rozdań.

3.8.4. Rozliczenie zakładu Blind

Zakład Blind ma stałą stawkę równą jednej jednostce Ante. W przypadku zwycięstwa gracza jego zysk zależy od kategorii uzyskanego układu. Dla układów słabszych od strita Blind jest zwracany. Dla strita lub silniejszego układu stosowana jest tabela wypłat przedstawiona w tabeli 3.10.

Tabela 3.10. Tabela wypłat zakładu Blind

Układ gracza	Wypłata	Zysk netto
Wysoka karta	zwrot	0
Jedna para	zwrot	0
Dwie pary	zwrot	0
Trójka	zwrot	0
Strit	1:1	1
Kolor	3:2	$\frac{3}{2}$
Full	3:1	3
Kareta	10:1	10
Poker	50:1	50
Poker królewski	500:1	500

Poker królewski jest w evaluatorze szczególnym przypadkiem pokera z asem jako najwyższą kartą. Nie stanowi osobnej kategorii układu, ale jest rozpoznawany podczas wyznaczania wypłaty Blind.

Tabela Blind jest stosowana wyłącznie wtedy, gdy gracz wygrał showdown. Przy remisie zakład jest zwracany, natomiast przy zwycięstwie krupiera pełna stawka Blind zostaje utracona.

3.8.5. Rozliczenie Ante i Play

Zakład Ante ma stałą wartość jednej jednostki. Przy remisie jest zwracany. Jeżeli krupier kwalifikuje się, Ante wygrywa albo przegrywa zgodnie z wynikiem showdownu. W przypadku braku kwalifikacji zakład jest zwracany.

Wartość zakładu Play jest określona przez moment podjęcia decyzji:

$$s_{Play} \in \{1, 2, 3, 4\}. \quad (3.27)$$

Zakład Play jest wypłacany 1:1, niezależnie od jego mnożnika. Przykładowo wygrany Play $4\times$ daje zysk netto równy czterem jednostkom. Przy porażce tracona jest pełna wartość zakładu, natomiast przy remisie cała stawka zostaje zwrócona.

3.8.6. Rozliczenie spasowania

Jeżeli gracz spasuje na riverze, nie dochodzi do showdownu. Ante i Blind zostają przegrane, natomiast Play nie został postawiony.

Rozliczenie folda ma zatem postać

$$S_{fold} = (n_{Ante} = -1, n_{Blind} = -1, n_{Play} = 0). \quad (3.28)$$

Łączna stawka wynosi dwie jednostki, a wynik netto jest równy

$$S_{net}^{fold} = -2. \quad (3.29)$$

W rozliczeniu Play zapisywana jest stawka równa zero. Pozwala to odróżnić zakład niepostawiony od postawionego zakładu, który został zwrócony.

3.8.7. Przykładowe rozliczenia

Pierwszy przykład zakłada, że gracz wykonał Play $4\times$, uzyskał kolor i pokonał kwalifikującego się krupiera. Poszczególne zyski netto wynoszą

$$n_{Ante} = 1, \quad n_{Blind} = \frac{3}{2}, \quad n_{Play} = 4. \quad (3.30)$$

Łączny wynik netto jest równy

$$S_{net} = 1 + \frac{3}{2} + 4 = \frac{13}{2}. \quad (3.31)$$

W drugim przykładzie gracz wykonał Play $2\times$ i wygrał showdown wysoką kartą przeciwko niekwalifikującemu się krupierowi. Ante oraz Blind są zwracane, a zysk pochodzi wyłącznie z Play:

$$S_{net} = 0 + 0 + 2 = 2. \quad (3.32)$$

Trzeci przykład przedstawia przypadek, w którym niekwalifikujący się krupier ma wyższą wysoką kartę niż gracz. Przy zakładzie Play $1\times$ Ante jest zwracane, ale Blind i Play zostają przegrane:

$$S_{net} = 0 - 1 - 1 = -2. \quad (3.33)$$

Przypadek ten pokazuje, że warunek kwalifikacji wpływa na rozliczenie Ante, lecz nie zastępuje porównania układów.

3.9. Funkcja nagrody i wynik epizodu

Mechanizm rozliczania zakładów został oddzielony od funkcji nagrody wykorzystywanej przez algorytmy uczenia przez wzmacnianie. Silnik domenowy odpowiada za poprawne wyznaczenie wyniku finansowego zgodnie z zasadami Ultimate Texas Hold'em. Funkcja nagrody będzie natomiast elementem warstwy integrującej silnik z agentem uczącym się.

Rozdzielenie tych dwóch odpowiedzialności pozwala wykorzystywać tę samą implementację zasad gry podczas eksperymentów z różnymi sposobami konstruowania sygnału nagrody. Zmiana funkcji nagrody nie wymaga dzięki temu modyfikowania mechanizmu showdownu, kwalifikacji krupiera ani tabel wypłat.

3.9.1. Rozliczenie finansowe a nagroda

Rozliczenie Settlement jest obiektem domenowym opisującym rzeczywisty wynik rozdania. Zawiera osobne wyniki zakładów Ante, Blind i Play oraz ich łączny wynik netto. Nie jest ono bezpośrednio nagrodą w rozumieniu uczenia przez wzmacnianie, ponieważ zawiera więcej informacji niż pojedyncza wartość skalarna oczekiwana przez większość algorytmów.

Funkcję nagrody można przedstawić jako odwzorowanie

$$R : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}, \quad (3.34)$$

które przypisuje wartość liczbową przejściu ze stanu s_t do s_{t+1} po wykonaniu akcji a_t .

W podstawowym wariancie źródłem nagrody terminalnej będzie łączny wynik netto rozdania:

$$R_{net}(S) = S_{net}, \quad (3.35)$$

gdzie S_{net} oznacza wartość `Settlement.total_net_profit`.

Wynik netto został wybrany zamiast zwrotu brutto, ponieważ nie zawiera zwracanych stawek początkowych. Przykładowo zwrócony zakład ma dodatni zwrot brutto równy jego stawce, ale nie oznacza osiągnięcia zysku. W reprezentacji netto taki przypadek otrzymuje wartość zero.

3.9.2. Podstawowa nagroda terminalna

Pojedyncze rozdanie stanowi jeden epizod. W podstawowej funkcji nagrody wszystkie przejścia pośrednie otrzymują wartość zero, natomiast końcowe przejście otrzymuje łączny wynik netto rozdania:

$$r_t = \begin{cases} 0, & s_{t+1} \notin \mathcal{S}_{terminal}, \\ S_{net}, & s_{t+1} \in \mathcal{S}_{terminal}. \end{cases} \quad (3.36)$$

Oznacza to, że akcja CHECK nie generuje natychmiastowej nagrody. Jej konsekwencje są uwzględniane dopiero w wyniku całego rozdania. Wykonanie zakładu Play prowadzi bezpośrednio do showdownu, dlatego rezultat finansowy jest zwracany w tym samym kroku, w którym agent wybrał akcję BET.

Spasowanie również tworzy nagrodę terminalną. Ponieważ gracz traci Ante i Blind, a zakład Play nie został postawiony, podstawowa nagroda wynosi

$$r_{fold} = -2. \quad (3.37)$$

Dla przykładowej wygranej z zyskiem netto równym $\frac{13}{2}$ nagroda terminalna przyjmuje wartość

$$r_{terminal} = \frac{13}{2}. \quad (3.38)$$

Wewnętrzne rozliczenia są reprezentowane dokładnie za pomocą typu `Fraction`. Adapter środowiska będzie przekształcał końcową wartość na typ liczbowy wymagany przez bibliotekę uczenia, na przykład `float`, dopiero na granicy interfejsu.

3.9.3. Zwrot epizodyczny

Zwrot uzyskany przez agenta od chwili t można zapisać jako

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}, \quad (3.39)$$

gdzie T oznacza koniec epizodu, a γ jest współczynnikiem dyskontowania.

Ponieważ w podstawowym wariancie wszystkie nagrody pośrednie są równe zero, dla $\gamma = 1$ całkowity zwrot epizodu jest równy wynikowi netto rozdania:

$$G_0 = S_{net}. \quad (3.40)$$

Zastosowanie współczynnika $\gamma < 1$ powodowałoby, że ta sama wartość terminalna byłaby silniej dyskontowana w rozdaniach zawierających więcej decyzji. Strategia kończąca rozdanie przed flopem mogłaby więc otrzymywać inną efektywną wartość niż strategia osiągająca identyczny wynik po dwóch akcjach CHECK. Z tego względu wpływ współczynnika dyskontowania musi zostać uwzględniony podczas konfiguracji eksperymentów.

3.9.4. Brak nagrody za nielegalną akcję

Nielegalna akcja nie stanowi poprawnego przejścia środowiska. Silnik odrzuca ją przez zgłoszenie wyjątku, zanim zostanie zmieniony stan lub pobrane zostaną kolejne karty. Nie jest zatem tworzona nagroda odpowiadająca takiej decyzji.

W podstawowej integracji z agentami wybór nielegalnych akcji będzie ograniczany za pomocą zbioru legalnych decyzji lub maski akcji. Silnik pozostanie końcowym mechanizmem walidującym, natomiast kod sterujący eksperymentem powinien traktować wystąpienie nielegalnej decyzji jako błąd implementacji agenta, a nie jako normalny element trajektorii uczenia.

Nie przyjęto automatycznej kary za nielegalną akcję. Wprowadzenie takiej wartości mieszałoby dwa różne zjawiska: jakość strategii pokerowej oraz poprawność techniczną komponentu wybierającego decyzje.

3.9.5. Możliwość stosowania alternatywnych funkcji nagrody

Jednym z celów pracy jest zbadanie wpływu funkcji nagrody na jakość uzyskiwanej strategii. Z tego względu adapter zostanie zaprojektowany tak, aby sposób konstruowania nagrody nie był zapisany bezpośrednio w silniku domenowym.

Oprócz surowego wyniku netto możliwe jest rozważenie między innymi:

- nagrody znormalizowanej względem liczby postawionych jednostek,
- nagrody ograniczonej do ustalonego przedziału,
- nagrody opisującej jedynie znak wyniku rozdania,
- nagrody pośredniej wspomagającej proces uczenia.

Przykładową nagrodę znormalizowaną można zapisać jako

$$R_{normalized}(S) = \frac{S_{net}}{S_{stake}}, \quad (3.41)$$

gdzie $S_{stake} > 0$ oznacza całkowitą wartość zakładów postawionych w rozdaniu.

Wariant oparty wyłącznie na znaku wyniku mógłby przyjmować postać

$$R_{sign}(S) = \begin{cases} 1, & S_{net} > 0, \\ 0, & S_{net} = 0, \\ -1, & S_{net} < 0. \end{cases} \quad (3.42)$$

Takie przekształcenie ogranicza wpływ rzadkich, bardzo wysokich wypłat, ale jednocześnie usuwa informację o rzeczywistej wartości finansowej decyzji. Przykładowo wygrana jednej jednostki i wygrana wynikająca z pokera królewskiego otrzymałyby identyczną wartość.

Nagroda pośrednia mogłaby dostarczać agentowi dodatkowy sygnał przed zakończeniem rozdania. Jej zastosowanie wymaga jednak ostrożności, ponieważ niepoprawnie zaprojektowane kształtowanie nagrody może skłonić model do optymalizacji pomocniczego kryterium zamiast długoterminowego wyniku finansowego.

Dokładny zestaw badanych wariantów funkcji nagrody oraz ich parametry zostaną przedstawione w rozdziale opisującym metodykę eksperymentów. Podstawowym punktem odniesienia pozostanie nagroda terminalna równa wynikowi netto rozdania.

3.10. Interfejs silnika i integracja z Gymnasium

Silnik domenowy udostępnia niewielki interfejs publiczny, za pomocą którego można rozpocząć rozdanie, pobrać obserwację, wykonać decyzję oraz odczytać wynik zakończzonego epizodu. Interfejs nie zależy od konkretnego algorytmu uczenia ani od biblioteki Gymnasium.

Centralnym elementem jest klasa `UTHGame`. Pojedyncza instancja przechowuje stan aktualnego rozdania, źródło kart oraz opcjonalną historię decyzji. Po zakończeniu rozdania ta sama instancja może zostać wykorzystana ponownie przez wywołanie metody `reset()`.

3.10.1. Tworzenie silnika

Konstruktor klasy `UTHGame` przyjmuje dwa opcjonalne parametry:

- `seed` – ziarno nadrzędnego generatora liczb pseudolosowych,
- `record_history` – informację, czy należy rejestrować przebieg kolejnych decyzji.

Ziarno określa powtarzalny strumień talii generowanych dla kolejnych rozdawań.

Parametr nie ustala pojedynczej, niezmienniej talii dla całej instancji. Każde wywołanie `reset()` tworzy nową talię, której ziarno jest pobierane z generatora środowiska.

Rejestrowanie historii jest domyślnie wyłączone. Dzięki temu podstawowe wykonywanie dużej liczby rozdań nie wymaga przechowywania dodatkowych obiektów diagnostycznych.

3.10.2. Rozpoczęcie epizodu

Metoda `reset()` rozpoczyna nowe rozdanie i zwraca pierwszą obserwację w fazie PREFLOP. W standardowym użyciu środowisko samodzielnie tworzy przetasowaną talię. Metoda umożliwia również przekazanie własnego obiektu implementującego interfejs `CardSource`, na przykład deterministycznej talii `FixedDeck`.

Sygnaturę operacji można przedstawić jako

$$\text{reset} : \text{CardSource} \cup \{\text{None}\} \rightarrow \text{UTHObservation}. \quad (3.43)$$

Rozpoczęcie rozdania obejmuje:

1. wybór lub utworzenie źródła kart,
2. rozdanie dwóch kart graczowi i dwóch kart krupierowi,
3. utworzenie stanu PREFLOP,
4. wyzerowanie historii decyzji bieżącego rozdania,
5. utworzenie obserwacji gracza.

Reset jest wykonywany transakcyjnie. Silnik najpierw próbuje pobrać wszystkie karty potrzebne do utworzenia stanu początkowego, a dopiero po powodzeniu zapisuje nowy stan i źródło kart. Jeżeli przekazane źródło nie może dostarczyć wymaganych kart, poprzedni stan silnika nie zostaje zastąpiony stanem częściowo utworzonego rozdania.

3.10.3. Wykonanie decyzji

Metoda `step()` przyjmuje jedną wartość typu `Action` i wykonuje odpowiadające jej przejście. Jej uproszczona sygnatura ma postać

$$\text{step} : \text{Action} \rightarrow \text{StepResult}. \quad (3.44)$$

Wynikiem operacji jest obiekt `StepResult`, zawierający:

- obserwację po wykonaniu akcji,
- wynik rozdania, jeżeli osiągnięto stan terminalny,

- rozliczenie zakładów w stanie terminalnym,
- szczegóły showdownu, jeżeli rozdanie nie zakończyło się foldem.

Wynik kroku pośredniego zawiera jedynie kolejną obserwację. Pola dotyczące wyniku i rozliczenia pozostają wtedy nieokreślone. W kroku terminalnym obiekt zawiera komplet informacji niezbędnych do obliczenia nagrody i zarejestrowania wyniku eksperymentu.

Właściwość `terminated` obiektu `StepResult` informuje, czy wykonana akcja zakończyła rozdanie. Jej wartość jest wyznaczana na podstawie fazy zawartej w obserwacji.

3.10.4. Właściwości publiczne

Poza metodami `reset()` i `step()` silnik udostępnia właściwości przedstawione w tabeli 3.11.

Tabela 3.11. Publiczny interfejs klasy `UTHGame`

Element	Typ wyniku	Znaczenie
<code>reset()</code>	<code>UTHObservation</code>	Rozpoczyna nowe rozdanie i zwraca obserwację preflop.
<code>step(action)</code>	<code>StepResult</code>	Wykonuje legalną akcję i zwraca wynik przejścia.
<code>observation</code>	<code>UTHObservation</code>	Zwraca aktualne informacje widoczne dla agenta.
<code>state</code>	<code>RoundState</code>	Zwraca pełny stan wewnętrzny, przeznaczony głównie do testów i diagnostyki.
<code>is_started</code>	<code>bool</code>	Informuje, czy rozpoczęto co najmniej jedno rozdanie.
<code>history_enabled</code>	<code>bool</code>	Informuje, czy rejestrowanie historii jest aktywne.
<code>trace</code>	<code>RoundTrace</code> lub <code>None</code>	Zwraca zapis bieżącego rozdania, jeżeli historia jest włączona.

Właściwość `state` nie jest częścią interfejsu przeznaczonego do podejmowania decyzji przez agenta. Udostępniono ją na potrzeby testów, diagnostyki i analizy środowiska. Agent powinien otrzymywać wyłącznie wartość właściwości `observation` albo obserwację zwróconą przez `reset()` lub `step()`.

3.10.5. Rola adaptera `Gymnasium`

Silnik `UTHGame` nie implementuje bezpośrednio klasy środowiska `Gymnasium`. Zostanie on opakowany przez osobny adapter, którego zadaniem będzie przekształcenie obiektów domenowych na format oczekiwany przez algorytmy uczenia przez wzmocnienie.

Adapter nie będzie implementował zasad gry. Wszystkie przejścia, walidacja akcji, showdown i rozliczenia pozostaną odpowiedzialnością UTHGame. Warstwa integracyjna będzie odpowiadała za:

- numeryczne kodowanie obserwacji,
- odwzorowanie indeksów akcji na wartości Action,
- utworzenie maski legalnych akcji,
- przekształcenie wyniku netto w nagrodę,
- utworzenie wartości *terminated* i *truncated*,
- przygotowanie dodatkowych danych w słowniku *info*.

Taki podział pozwala zmieniać sposób reprezentacji obserwacji oraz funkcję nagrody bez modyfikowania silnika domenowego.

3.10.6. Odwzorowanie metody reset

Metoda `reset()` adaptera będzie zwracała parę

$$(o_0, info_0), \quad (3.45)$$

gdzie o_0 oznacza numeryczną reprezentację pierwszej obserwacji, a $info_0$ zawiera dodatkowe dane niewchodzące bezpośrednio w skład reprezentacji stanu.

Przebieg operacji będzie następujący:

1. adapter rozpoczyna nowy epizod w UTHGame,
2. silnik zwraca domenowy obiekt UTHObservation,
3. wybrany encoder przekształca obserwację do postaci numerycznej,
4. adapter tworzy maskę akcji i dane pomocnicze,
5. zakodowana obserwacja i słownik *info* są zwracane agentowi.

Ziarno przekazane do adaptera będzie wykorzystywane do inicjalizacji powtarzalnego przebiegu środowiska. Szczegóły propagowania ziarna zostaną ustalone podczas implementacji adaptera, aby zachować zgodność pomiędzy generatorem Gymnasium i generatorem silnika domenowego.

3.10.7. Odzworowanie metody step

Metoda `step()` adaptera będzie zwracała pięć wartości:

$$(o_{t+1}, r_t, terminated_t, truncated_t, info_t). \quad (3.46)$$

Dla pojedynczego przejścia adapter wykona następujące operacje:

1. przekształci indeks akcji na wartość typu `Action`,
2. przekaże akcję do metody `UTHGame.step()`,
3. zakoduje otrzymaną obserwację,
4. wyznaczy nagrodę na podstawie rozliczenia,
5. odczyta informację o zakończeniu rozdania,
6. utworzy słownik danych pomocniczych.

Flaga `terminated` będzie przyjmowała wartość prawdziwą po naturalnym zakończeniu rozdania przez showdown albo fold. Podstawowe środowisko nie nakłada zewnętrznego limitu kroków, dlatego `truncated` będzie domyślnie fałszywe. Najdłuższe poprawne rozdanie zawiera trzy decyzje, więc samo środowisko nie wymaga dodatkowego limitu czasu.

Rozróżnienie tych dwóch flag jest istotne. `terminated` oznacza osiągnięcie rzeczywistego stanu końcowego procesu decyzyjnego, natomiast `truncated` służy do oznaczania przerwania epizodu przez warunek zewnętrzny wobec dynamiki gry.

3.10.8. Planowana przestrzeń akcji

Pełna przestrzeń akcji adaptera będzie stałą przestrzenią dyskretną o liczności sześciu:

$$\mathcal{A}_{gym} = \{0, 1, 2, 3, 4, 5\}. \quad (3.47)$$

Każdy indeks zostanie jednoznacznie przypisany do jednej wartości `Action`. Ponieważ liczba legalnych akcji zmienia się pomiędzy fazami, adapter będzie dodatkowo konstruował binarną maskę

$$M_t = (m_0, m_1, \dots, m_5), \quad m_i \in \{0, 1\}. \quad (3.48)$$

Wartość $m_i = 1$ będzie oznaczała, że akcja o indeksie i jest legalna w bieżącej obserwacji. Stała liczność przestrzeni umożliwi zastosowanie jednej warstwy wyjściowej modelu we wszystkich fazach rozdania.

3.10.9. Planowana przestrzeń obserwacji

Ostateczna postać przestrzeni obserwacji nie została jeszcze ustalona, ponieważ porównanie reprezentacji stanu stanowi jeden z elementów badawczych pracy. Każdy wariant będzie jednak kodował ten sam dozwolony zakres informacji wynikający z UTHObservation.

Rozważane reprezentacje mogą obejmować:

- zakodowane identyfikatory kart gracza i kart wspólnych,
- wektor binarny wskazujący obecność poszczególnych kart,
- osobne kodowanie rang i kolorów,
- cechy pochodne opisujące aktualną siłę ręki,
- kod fazy oraz maskę legalnych akcji.

Niezależnie od wariantu obserwacja będzie miała stały rozmiar. Brakujące karty wspólne w fazach PREFLOP i FLOP zostaną reprezentowane przez ustaloną wartość pustą albo odpowiednie wyzerowanie wektora.

Oddzielenie encodera od adaptera pozwoli porównywać różne reprezentacje przy zachowaniu identycznej dynamiki środowiska, sekwencji rozdań i sposobu rozliczania zakładów.

3.10.10. Dane pomocnicze

Słownik info będzie przeznaczony dla danych diagnostycznych i wyników, które nie powinny być bezpośrednią częścią obserwacji modelu. Może on zawierać między innymi:

- maskę legalnych akcji,
- wynik rozdania,
- łączną wartość postawionych zakładów,
- wynik netto,
- osobne rozliczenia Ante, Blind i Play,
- informację o kwalifikacji krupiera,
- szczegóły showdownu w zakończonym epizodzie.

Dane terminalne zostaną umieszczone w info dopiero po zakończeniu rozdania. Nie będą one wykorzystywane jako element wejścia agenta przed podjęciem decyzji.

3.10.11. Porównanie interfejsów

Relację pomiędzy obecnym interfejsem domenowym a planowanym adapterem przedstawiono w tabeli 3.12.

Tabela 3.12. Odzworowanie interfejsu domenowego na Gymnasium

Element	Silnik domenowy	Adapter Gymnasium
Rozpoczęcie epizodu	UTHObservation	Zakodowana obserwacja oraz info.
Akcja	Wartość Action	Indeks ze stałej przestrzeni dyskretnej.
Wynik kroku	StepResult	Obserwacja, nagroda, terminated, truncated i info.
Nagroda	Brak bezpośredniej wartości	Obliczana przez wybraną funkcję nagrody.
Legalność akcji	frozenset[Action]	Maska binarna oraz walidacja silnika.
Stan ukryty	RoundState	Nie jest przekazywany agentowi.
Rozliczenie	Dokładne wartości Fraction	Wartości przekształcone do formatu używanego przez eksperyment.

Adapter będzie zatem pełnił rolę warstwy translacyjnej. Nie będzie drugim silnikiem gry ani miejscem implementacji dodatkowych reguł. Takie podejście zapobiega powstaniu dwóch niezależnych implementacji UTH, które mogłyby zwracać różne wyniki dla tego samego rozdania.

3.11. Rejestrowanie przebiegu i walidacja środowiska

Poprawność środowiska ma bezpośredni wpływ na wyniki eksperymentów. Błąd w kolejności rozdawania kart, rozliczaniu zakładów albo udostępnianiu obserwacji mógłby zostać błędnie zinterpretowany jako zachowanie wyuczone przez agenta. Z tego względu implementację uzupełniono o opcjonalny mechanizm rejestrowania przebiegu rozdania oraz zestaw testów weryfikujących zarówno pojedyncze komponenty, jak i pełne ścieżki rozgrywki.

3.11.1. Opcjonalne rejestrowanie historii

Rejestrowanie historii jest aktywowane parametrem `record_history` przekazywanym podczas tworzenia obiektu `UTHGame`. Domyślnie mechanizm pozostaje wyłączony, aby standardowe wykonywanie dużej liczby epizodów nie wymagało tworzenia dodatkowych obiektów diagnostycznych.

Po włączeniu historii każde poprawnie rozpoczęte rozdanie otrzymuje dodatni, rosnący identyfikator `round_id`. Lista decyzji jest zerowana przy każdym poprawnym wywołaniu metody `reset()`.

Właściwość `trace` zwraca obiekt `RoundTrace`. Jeżeli historia jest wyłączona albo nie rozpoczęto jeszcze rozdania, jej wartością jest `None`.

Mechanizm historii nie stanowi pamięci agenta i nie jest częścią obserwacji. Jest przeznaczony do:

- diagnozowania nieoczekiwanych decyzji,
- odtwarzania przebiegu wybranych rozdań,
- analizy działania agentów bazowych i uczących się,
- zapisywania przykładów błędów podczas eksperymentów,
- weryfikowania zgodności decyzji z legalną przestrzenią akcji.

3.11.2. Rekord pojedynczej decyzji

Pojedynczą decyzję reprezentuje niemodyfikowalny obiekt `DecisionRecord`. Zawiera on:

- obserwację dostępną agentowi przed wykonaniem decyzji,
- wybraną akcję.

Rekord można przedstawić jako parę

$$d_t = (O_t, A_t), \quad (3.49)$$

gdzie O_t oznacza obserwację przed wykonaniem akcji, a A_t oznacza decyzję agenta.

Zapisanie obserwacji sprzed przejścia jest istotne, ponieważ pozwala odtworzyć informacje, na podstawie których agent rzeczywiście podjął decyzję. Zapis obserwacji po wykonaniu akcji prowadziłby do niejednoznaczności: mógłby zawierać karty odkryte dopiero jako skutek tej decyzji.

Podczas tworzenia rekordu sprawdzane jest, czy:

- obserwacja jest obiektem `UTHObservation`,
- akcja jest wartością typu `Action`,
- obserwacja nie jest terminalna,
- akcja należy do zbioru legalnego dla zapisanej obserwacji.

Dzięki temu historia nie może zawierać decyzji wykonanej po zakończeniu rozdań ani akcji sprzecznej z fazą zapisaną w obserwacji.

3.11.3. Ślad całego rozdania

Obiekt RoundTrace reprezentuje niemodyfikowalny zapis aktualnego przebiegu rozdania. Zawiera:

- dodatni identyfikator rozdania,
- uporządkowaną krotkę obiektów DecisionRecord,
- aktualny pełny stan RoundState.

Dla rozdania zawierającego n decyzji jego ślad można zapisać jako

$$\tau = (id, d_0, d_1, \dots, d_{n-1}, S_n). \quad (3.50)$$

Obserwacje znajdujące się w rekordach decyzji zawierają wyłącznie informacje dostępne agentowi. Końcowy element S_n jest natomiast pełnym stanem diagnostycznym i może zawierać karty krupiera, karty spalone oraz rozliczenie rozdania.

Z tego powodu RoundTrace nie może być przekazywany agentowi jako obserwacja. Jest to obiekt przeznaczony dla kodu eksperymentalnego, testów i narzędzi analitycznych.

Ślad udostępnia również właściwości pomocnicze:

- `completed` – informację, czy osiągnięto stan terminalny,
- `outcome` – końcowy wynik rozdania,
- `settlement` – rozliczenie Ante, Blind i Play,
- `showdown` – szczegóły porównania rąk, jeżeli do niego doszło.

Przed zakończeniem rozdania właściwości związane z wynikiem mogą zwracać wartość `None`. Po foldzie dostępne jest rozliczenie i wynik, ale nie występują szczegóły showdownu.

3.11.4. Transakcyjne zapisywanie decyzji

Metoda `step()` najpierw zapisuje bieżącą obserwację w zmiennej lokalnej, następnie próbuje wykonać właściwe przejście, a dopiero po jego powodzeniu dodaje rekord do historii.

Kolejność operacji ma postać

$$O_t \rightarrow \text{validate}(A_t) \rightarrow \text{transition}(S_t, A_t) \rightarrow S_{t+1} \rightarrow \text{record}(O_t, A_t). \quad (3.51)$$

Jeżeli walidacja lub wykonywanie przejścia zakończy się wyjątkiem, etap `record` nie zostaje osiągnięty. Historia zawiera zatem wyłącznie decyzje, dla których środowisko utworzyło poprawny kolejny stan.

Takie zachowanie zabezpiecza między innymi przed zapisaniem:

- akcji nielegalnej w aktualnej fazie,
- decyzji po zakończeniu rozdania,
- akcji, podczas której źródło kart zwróciło niepoprawną wartość,
- niedokończonego przejścia wynikającego z pustej talii.

Ta sama zasada jest stosowana podczas resetowania środowiska. Nowy stan, źródło kart, identyfikator rozdania i pusta historia są zapisywane dopiero po poprawnym rozdaniu wszystkich czterech kart początkowych.

3.11.5. Testy deterministyczne

Znaczna część testów pełnych rozdań wykorzystuje deterministyczne źródło `FixedDeck`. Jawne określenie kolejności kart pozwala przygotować scenariusz, którego wynik nie zależy od generatora pseudolosowego.

Deterministyczny scenariusz może określać kolejno:

$$P_1, D_1, P_2, D_2, B_1, F_1, F_2, F_3, B_2, T, R, \quad (3.52)$$

gdzie P i D oznaczają karty własne gracza i krupiera, B karty spalone, F karty flopa, T turn, a R river.

Tak przygotowane źródło umożliwia kontrolowanie jednocześnie:

- kart obu stron,
- końcowego układu gracza i krupiera,
- kwalifikacji krupiera,
- wyniku showdownu,
- wypłaty zakładu Blind,
- łącznego wyniku netto.

W przeciwieństwie do testu opartego wyłącznie na ziarnie `FixedDeck` jednoznacznie pokazuje w kodzie testowym, jakie karty mają zostać dobrane. Ułatwia to analizę nieudanego przypadku i ogranicza zależność testu od szczegółów algorytmu tasowania.

3.11.6. Poziomy testowania

Testy środowiska podzielono według odpowiedzialności sprawdzanych komponentów. Najważniejsze obszary przedstawiono w tabeli 3.13.

Tabela 3.13. Główne obszary testowania środowiska UTH

Obszar	Przykładowe przypadki	Weryfikowana własność
Model kart i talia	Pełna talia, brak duplikatów, pusta talia, stałe ziarno	Poprawność źródła kart i kolejności dobierania.
Evaluator	Kategorie układów, remisy, niski strit, najlepsze pięć z siedmiu	Jednoznaczne porównywanie rąk.
Modele stanu	Liczba kart w fazach, duplikaty, pola terminalne	Zachowanie niezmienników <code>RoundState</code> i obserwacji.
Rozdawanie	Karty początkowe, flop, turn i river, karty spalone	Poprawna kolejność i dozwolone przejścia faz.
Legalność akcji	Akcje dozwolone i zabronione w każdej fazie	Odrzucanie decyzji sprzecznych z zasadami.
Showdown	Wygrana gracza, wygrana krupiera, remis	Poprawne wyznaczenie <code>RoundOutcome</code> .
Rozliczenia	Kwalifikacja, tabela Blind, różne mnożniki Play, fold	Poprawność wyniku Ante, Blind, Play i wyniku łącznego.
Pełne rozdania	Wszystkie ścieżki maszyny stanów	Integracja komponentów od <code>reset()</code> do stanu terminalnego.
Historia	Jedna, dwie i trzy decyzje, błędne przejście	Rejestrowanie wyłącznie zakończonych sukcesem decyzji.

Testy jednostkowe sprawdzają komponenty w izolacji, natomiast testy integracyjne wykonują kompletne rozdania. Samo poprawne przetestowanie evaluatora i modułu rozliczeń nie gwarantuje bowiem, że silnik przekaże im właściwe karty i mnożnik zakładu Play.

3.11.7. Walidowane niezmienniki

Poza sprawdzaniem przykładowych wyników testy weryfikują niezmienniki obowiązujące dla wszystkich poprawnych rozdań. Należą do nich między innymi:

- brak powtarzających się kart w stanie,
- dokładnie dwie karty własne każdej strony,
- liczba kart wspólnych zgodna z fazą,
- liczba kart spalonych zgodna z fazą,
- brak danych terminalnych w stanie pośrednim,
- obecność wyniku i rozliczenia w stanie terminalnym,

- brak showdownu po spasowaniu,
- dokładnie jeden zakład Play albo brak tego zakładu po foldzie,
- zgodność mnożnika Play z akcją kończącą część decyzyjną,
- zgodność wyniku RoundState, StepResult i ShowdownResult,
- brak kart krupiera i kart spalonych w obserwacji,
- brak zmiany stanu po odrzuconej akcji.

Walidacja nie ogranicza się zatem do testowania typowych scenariuszy zwycięstwa i porażki. Uwzględnia również przypadki brzegowe oraz próby utworzenia stanów, które naruszałby reguły domenowe.

3.11.8. Pokrycie kodu

Zestaw testów uruchamiany jest za pomocą biblioteki pytest. Podczas prac nad silnikiem analizowano zarówno pokrycie instrukcji, jak i pokrycie rozgałęzień. Pozwoliło to wykryć nieprzetestowane ścieżki walidacji, przypadki brzegowe ewaluatora oraz alternatywne sposoby rozliczania zakładów.

Na etapie zakończenia implementacji podstawowego silnika uzyskano pełne pokrycie instrukcji i rozgałęzień dla objętych pomiarem modułów. Wynik pokrycia nie stanowi samodzielnego dowodu poprawności implementacji. Informuje jedynie, że testy wykonały wszystkie mierzone linie i gałęzie programu. Z tego względu został uzupełniony testami opartymi na konkretnych scenariuszach domenowych oraz walidacją niezmienników.

Spis rysunków

3.1	Architektura i przepływ informacji w środowisku UTH	20
3.2	Oddzielenie pełnego stanu środowiska od obserwacji agenta	31
3.3	Maszyna stanów pojedynczego rozdania UTH	34

Spis tabel

2.1	Ranking układów pokerowych od najsilniejszego do najslabszego	10
2.2	Porównanie Texas Hold'em i Ultimate Texas Hold'em	11
2.3	Dostępne decyzje gracza w Ultimate Texas Hold'em	12
2.4	Tabela wypłat zakładu Trips przyjęta w modelu środowiska	13
2.5	Przykładowa tabela wypłat zakładu Trips w Ultimate Texas Hold'em . .	13
2.6	Przykładowe wartości przewagi kasyna dla wybranych gier kasynowych . .	14
3.1	Podział odpowiedzialności pomiędzy modułami środowiska UTH	19
3.2	Porównanie źródeł kart wykorzystywanych przez środowisko	23
3.3	Wartości wykorzystywane do rozstrzygania remisów	25
3.4	Liczba kart przechowywanych w stanie w poszczególnych fazach	28
3.5	Porównanie pełnego stanu środowiska i obserwacji agenta	30
3.6	Przejścia pomiędzy fazami środowiska UTH	33
3.7	Akcje dostępne w środowisku UTH	35
3.8	Rozliczenie zakładów po showdownie	39
3.9	Interpretacja wartości pojedynczego rozliczenia	40
3.10	Tabela wypłat zakładu Blind	40
3.11	Publiczny interfejs klasy UTHGame	47
3.12	Odwzorowanie interfejsu domenowego na Gymnasium	51
3.13	Główne obszary testowania środowiska UTH	55

Bibliografia

- [1] Tulalip Resort Casino, *Ultimate Texas Hold'em Game Rules*, https://www.everythingtulalip.com/wp-content/uploads/2024/04/ONEclub_0920_GameRules_RackCard_UltimateTexasHoldem-WEB.pdf, Dostęp: 2026-07-12.
- [2] Nevada Gaming Control Board, *Ultimate Texas Hold'em - Rules of Play*, https://www.gaming.nv.gov/siteassets/content/divisions/enforcement/rules-of-play/Ultimate_Texas_Hold_em.pdf, Dostęp: 2026-07-07.
- [3] W. of Odds, *Ultimate Texas Hold'em*, <https://wizardofodds.com/games/ultimate-texas-hold-em/>, Dostęp: 2026-07-07.
- [4] J. B. Maverick, *Understanding the House Edge: How Casinos Ensure Profit*, <https://www.investopedia.com/articles/personal-finance/110415/why-does-house-always-win-look-casino-profitability.asp>, Dostęp: 2026-07-12.
- [5] Wizard of Odds, *What is the House Edge?* <https://wizardofodds.com/gambling/house-edge/>, Dostęp: 2026-07-12.